UNIVERSITAT POLITÈCNICA DE VALÈNCIA

etsinf
Escola Tècnica Superior d'Enginyeria Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universitat Politècnica de València

Ingeniería inversa del sistema de alquiler de vehículos
Práctica 3 · Proyecto A7

Mantenimiento y Evolución del Software

Grado en Ingeniería Informática

Integrantes Zhijian Wu Wang
Jorge Carvajal Lara
Urtzi Etxegarai Taberna
Laura Lázaro Sartí
Steve Maxwelle Nembot Nembot

Curso 2026–2027

Índice general

1 Regeneración de la información de análisis 1
2 Regeneración de la información de diseño 2
2.1 Crear cliente 2
2.1.1 Diagrama de clases 2
2.1.2 Diagrama de secuencia 2
2.2 Entregar vehículo reservado 3
2.2.1 Diagrama de clases 3
2.2.2 Diagrama de secuencia 11
3 Mapa de la interfaz gráfica de usuario 36
4 Declaración de diligencia en el uso de la IA 37
4.1 Ejercicios 2.2.1 y 2.2.2 37
4.2 Aportaciones del resto del equipo 37

Índice de figuras

Figura 1. Vista general del diagrama de clases EJ2.2.1..... Content not found
Figura 2. Vista de presentación EJ2.2.1a..... Content not found
Figura 3. Vista del dominio EJ2.2.1b.....Content not found
Figura 4. Vista de acceso a datos EJ2.2.1c..... Content not found
Figura 5. Vista de DTO y excepción EJ2.2.1d..... Content not found
Figura 6. EJ2.2.2. Apertura de la ventana y eventos del caso de uso.....Content not found
Figura 7. ControladorEntregarVehiculoReserva.initialize..... Content not found
Figura 8. ControladorEntregarVehiculoReserva.popularTablaReservas..... Content not found
Figura 9. ControladorEntregarVehiculoReserva.listarCoches.....Content not found
Figura 10. ControladorEntregarVehiculoReserva.aceptar.....Content not found
Figura 11. AlquilerVehiculos.listarReservasSucursal..... Content not found
Figura 12. AlquilerVehiculos.getCliente.....Content not found
Figura 13. AlquilerVehiculos.getEntrega.....Content not found
Figura 14. AlquilerVehiculos.listarVehiculosCategoriaSucursal.....Content not found
Figura 15. AlquilerVehiculos.getEmpleado.....Content not found
Figura 16. AlquilerVehiculos.crearEntrega.....Content not found
Figura 17. AlquilerVehiculos.getReserva.....Content not found
Figura 18. Reserva.Reserva..... Content not found
Figura 19. Entrega.Entrega.....Content not found
Figura 20. Coche.Coche.....Content not found
Figura 21. DAL.getReservasSucursal..... Content not found
Figura 22. DAL.getCliente..... Content not found
Figura 23. DAL.getEntrega..... Content not found
Figura 24. DAL.obtenerCoches..... Content not found
Figura 25. DAL.getEmpleado.....Content not found
Figura 26. DAL.getCoche.....Content not found
Figura 27. DAL.getReserva.....Content not found
Figura 28. DAL.modificarCoche..... Content not found
Figura 29. DAL.crearEntrega.....Content not found

1 Regeneración de la información de análisis

La práctica reconstruye la información de análisis y diseño de la aplicación Java de alquiler de vehículos. El informe del equipo A7 reúne el modelo obtenido a partir del esquema SQL, el diseño de los dos casos de uso, el mapa de la interfaz gráfica y la declaración de uso de IA.

Falta completar. *Insertar el diagrama de clases de análisis EJ1, explicar cómo se ha obtenido a partir del esquema SQL y justificar las decisiones de transformación.*

2 Regeneración de la información de diseño

2.1 Crear cliente

2.1.1 Diagrama de clases

Figura X. Diagrama de clases EJ2.1.1

El diagrama del EJ2.1.1, que se corresponde con el primer caso de uso (Crear cliente), contiene 11 clases e interfaces de la vista del EJ2.1. Para identificarlas se siguió el código desde el manejador del botón Aceptar de la ventana Crear cliente hasta la sentencia de inserción en la base de datos, y se incluyeron las clases de dominio, el DTO y la excepción que intervienen.

Organización del modelo

Presentación. ControladorPrincipal abre la ventana cuando el usuario elige la opción Crear cliente del menú. ControladorCrearCliente hereda de ControladorCasoDeUso, que se encarga de la inicialización común de las ventanas. El controlador específico valida el formulario y delega el alta en AlquilerVehiculos.

Dominio y lógica. AlquilerVehiculos es el punto de entrada de la lógica. Mantiene en memoria la colección de clientes, indexada por DNI, y ofrece getCliente, que se usa para comprobar si el DNI ya existe, y crearCliente, que construye el Cliente, lo añade a la colección y solicita su almacenamiento.

Acceso a datos. AlquilerVehiculos accede a la persistencia a través de DAL, que delega en la interfaz IClienteDAO. Su implementación, ClienteDAOImp, usa ConnectionManager para ejecutar las sentencias SQL de consulta e inserción. Las capas intercambian los datos del cliente mediante ClienteDTO, y los errores de persistencia se representan con DAOExcepcion, que llega hasta el manejador de Aceptar.

Relaciones. ControladorCrearCliente hereda de ControladorCasoDeUso y ClienteDAOImp implementa IClienteDAO. AlquilerVehiculos se asocia con Cliente (colección de clientes) y con DAL; DAL se asocia con IClienteDAO, y ClienteDAOImp con ConnectionManager. El resto de relaciones son dependencias de uso: parámetros de métodos, creación de objetos o captura de excepciones.

Alcance y representación

El diagrama se ha exportado directamente desde BoUML. El diagrama editable se encuentra en A7.EJ2y3 (vista EJ2.1). AlquilerVehiculos y DAL contienen también colecciones y DAO de otras funcionalidades (reservas, coches, entregas…), pero no intervienen en el alta de un cliente y no se incluyen. Los tipos de JavaFX y del JDK forman parte del entorno de ejecución.

Clases de presentación

Elemento	Responsabilidad en el caso de uso
ControladorPrincipal	Controlador de la ventana principal. Cuando el usuario selecciona Crear cliente en el menú, su manejador solicita la inicialización del caso de uso y muestra su ventana.
ControladorCasoDeUso	Clase abstracta con los pasos comunes para abrir un caso de uso. Carga el FXML, obtiene el controlador de la vista y le pasa la ventana y el controlador principal.
ControladorCrearCliente	Controlador de la ventana Crear cliente. Al pulsar Aceptar valida los datos del formulario, comprobando también que el DNI no exista, y llama a AlquilerVehiculos para crear el cliente. Si todo va bien cierra la ventana y captura las DAOExcepcion que se propagan.

Lógica y dominio

Elemento	Responsabilidad en el caso de uso
AlquilerVehiculos	Fachada de la lógica de negocio (singleton). Guarda en memoria la colección de clientes. getCliente busca un DNI en memoria y, si no está, en la base de datos; crearCliente construye el Cliente, lo añade a la colección y envía su ClienteDTO a DAL.
Cliente	Objeto de dominio que representa al cliente: DNI, nombre y apellidos, dirección, población, código postal, fecha del carnet de conducir y datos de la tarjeta (dígitos, mes, año, CVC y tipo). Se construye a partir de sus datos o de un ClienteDTO.

Persistencia, datos intercambiados y excepción

Elemento	Responsabilidad en el caso de uso
DAL	Punto de entrada a la capa de acceso a datos (singleton). Crea los DAO y delega en ellos; en este caso, crearCliente y getCliente delegan en IClienteDAO y trabajan con ClienteDTO.

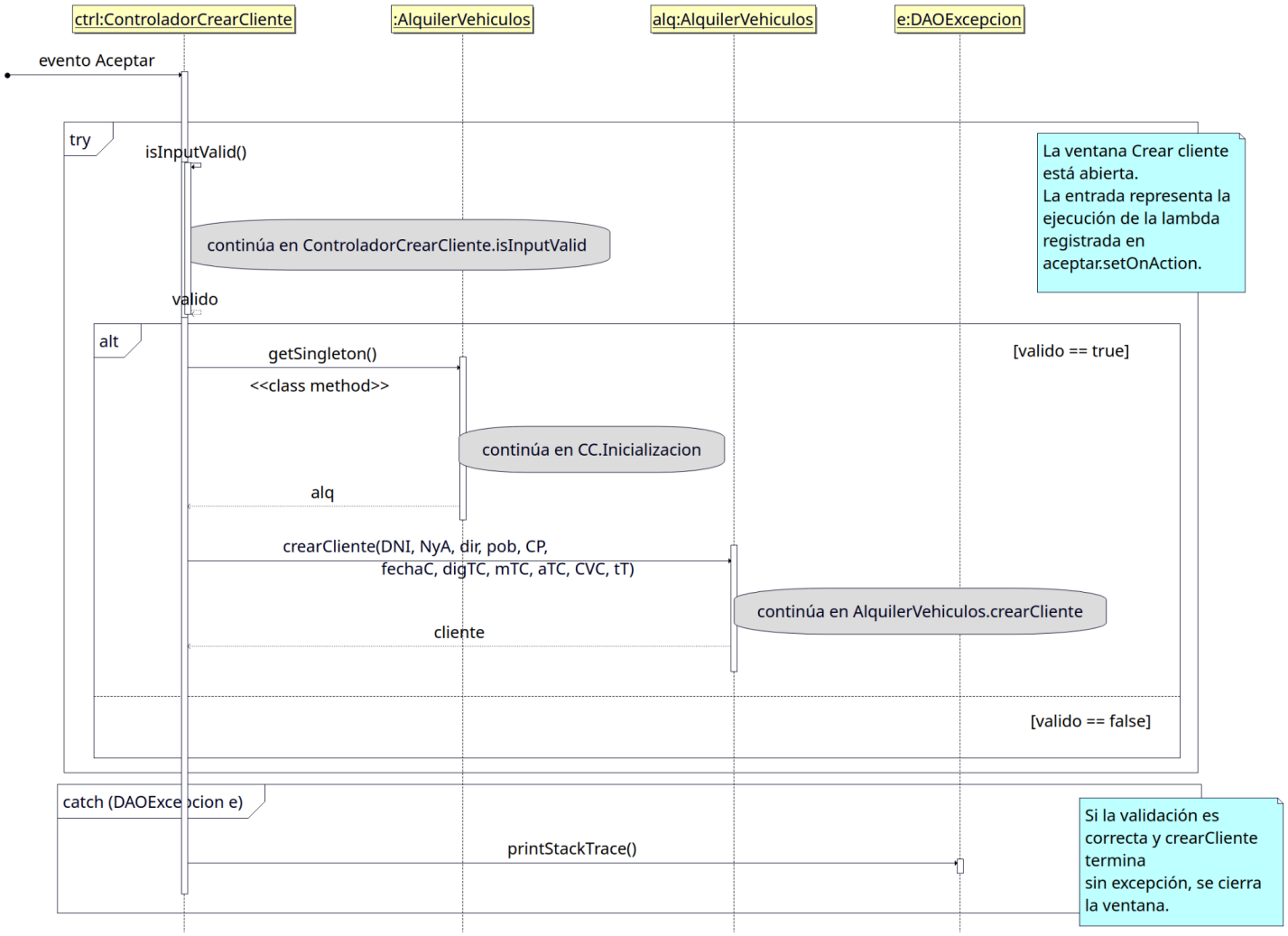
Elemento	Responsabilidad en el caso de uso
IClienteDAO	Define las operaciones de persistencia de clientes: buscar por DNI y crear.
ClienteDAOImp	Implementa IClienteDAO. Construye y ejecuta, mediante ConnectionManager, la consulta por DNI y la inserción del cliente, y convierte las SQLException en DAOExcepcion.
ConnectionManager	Abre y cierra la conexión con la base de datos (HSQLDB) y ejecuta las sentencias SQL que le pasan las implementaciones de los DAO.
ClienteDTO	Transfiere los datos de un cliente entre la lógica y la persistencia. Se construye a partir de un Cliente al crearlo y sirve para reconstruir el Cliente al consultarlo.
DAOExcepcion	Excepción propia que representa un error de acceso a datos. Se lanza en ClienteDAOImp, se propaga por DAL y AlquilerVehiculos y se captura en el manejador de Aceptar.

2.1.2 Diagrama de secuencia

El objetivo de estos diagramas es representar las interacciones entre los objetos que intervienen en la creación de un cliente, a partir del código existente. La secuencia comienza con la ventana Crear Cliente abierta, cuando el usuario pulsa el botón Aceptar.

Para facilitar su lectura, la secuencia se ha dividido en un diagrama principal y varios diagramas auxiliares. Cada continuación desarrolla la llamada indicada; no representa una segunda ejecución del método. Se omiten las invocaciones normales a bibliotecas de Java y se conservan las relacionadas con el tratamiento de errores. Las comprobaciones y asignaciones locales se explican mediante notas.

1. Diagrama principal: EJ2.1.2



El punto de entrada es la ejecución de la expresión lambda registrada mediante aceptar.setOnAction(...) en ControladorCrearCliente. El manejador invoca primero isInputValid() para comprobar los datos introducidos.

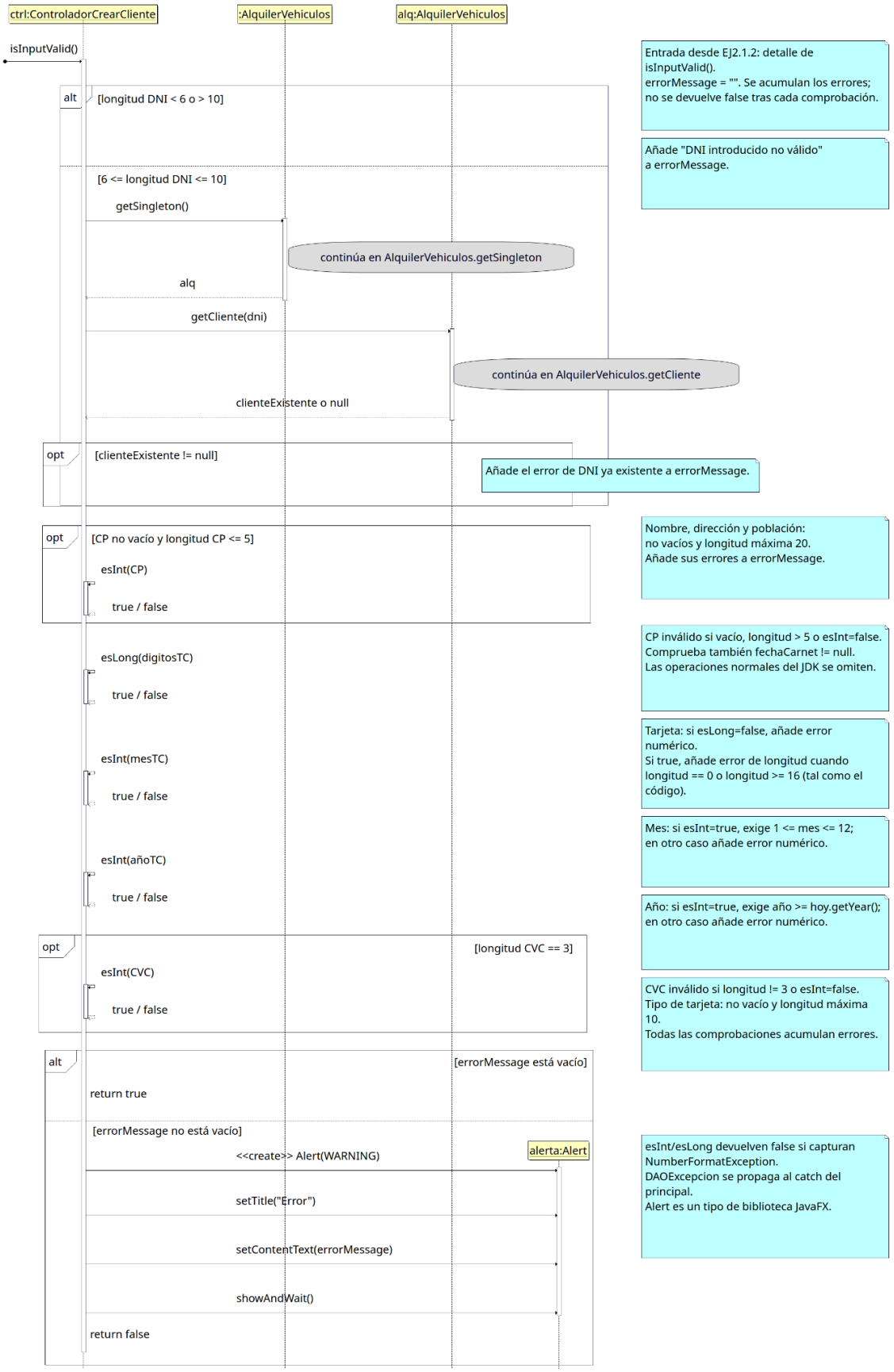
Si la validación devuelve true, se obtienen los datos del formulario y se realizan las conversiones necesarias. A continuación, se obtiene la instancia de AlquilerVehiculos

mediante getSingleton() y se invoca su método crearCliente(...). Cuando la operación finaliza sin excepciones, se cierra la ventana. Aunque crearCliente(...) devuelve un objeto Cliente, el manejador no utiliza ese valor.

Si la validación devuelve false, no se ejecuta la creación y la ventana permanece abierta. Si durante la validación o la creación se propaga una DAOExcepcion, el manejador la captura y ejecuta printStackTrace(). En este camino no se alcanza el cierre normal de la ventana.

(La etiqueta “Continua en CC.Inicialización” es en realidad “Continua en AlquilerVehiculo.getSIngleton).

2. ControladorCrearCliente.isInputValid


ctrl:ControladorCrearCliente
:AlquilerVehiculos
alq:AlquilerVehiculos
isInputValid()
Entrada desde EJ2.1.2: detalle de isInputValid().
errorMessage = "". Se acumulan los errores; no se devuelve false tras cada comprobación.
alt
[longitud DNI < 6 o > 10]
Añade "DNI introducido no válido" a errorMessage.
[6 <= longitud DNI <= 10]
getSingleton()
continúa en AlquilerVehiculos.getSingleton
alq
getCliente(dni)
continúa en AlquilerVehiculos.getCliente
clienteExistente o null
opt
[clienteExistente != null]
Añade el error de DNI ya existente a errorMessage.
opt
[CP no vacío y longitud CP <= 5]
Nombre, dirección y población: no vacíos y longitud máxima 20. Añade sus errores a errorMessage.
esInt(CP)
true / false
esLong(digitosTC)
true / false
CP inválido si vacío, longitud > 5 o esInt=false. Comprueba también fechaCarnet != null. Las operaciones normales del JDK se omiten.
esInt(mesTC)
true / false
Tarjeta: si esLong=false, añade error numérico. Si true, añade error de longitud cuando longitud == 0 o longitud >= 16 (tal como el código).
esInt(añoTC)
true / false
Mes: si esInt=true, exige 1 <= mes <= 12; en otro caso añade error numérico.
opt
[longitud CVC == 3]
Año: si esInt=true, exige año >= hoy.getYear(); en otro caso añade error numérico.
esInt(CVC)
true / false
CVC inválido si longitud != 3 o esInt=false. Tipo de tarjeta: no vacío y longitud máxima 10. Todas las comprobaciones acumulan errores.
alt
[errorMessage está vacío]
return true
[errorMessage no está vacío]
<<create>> Alert(WARNING)
alerta:Alert
esInt/esLong devuelven false si capturan NumberFormatException. DAOExcepcion se propaga al catch del principal. Alert es un tipo de biblioteca JavaFX.
setTitle("Error")
setContentText(errorMessage)
showAndWait()
return false

Este método inicializa una cadena errorMessage y acumula en ella los errores detectados. Las comprobaciones no devuelven false inmediatamente después de encontrar un campo incorrecto.

En primer lugar, se comprueba que la longitud del DNI esté comprendida entre 6 y 10 caracteres. Solo si cumple esta condición se obtiene el singleton de AlquilerVehiculos y se llama a getCliente(dni) para comprobar si ya existe. Si la búsqueda devuelve un cliente, se añade el error de DNI duplicado.

Después se comprueban los siguientes datos:

- Nombre y apellidos, dirección y población: no vacíos y con longitud máxima de 20 caracteres.

- Código postal: no vacío, longitud máxima de 5 caracteres y convertible a entero.

- Fecha de obtención del carnet: debe estar informada.

- Número de tarjeta: convertible a long y con longitud inferior a 16 caracteres.

- Mes de la tarjeta: convertible a entero y comprendido entre 1 y 12.

- Año de la tarjeta: convertible a entero y no anterior al año almacenado en hoy.

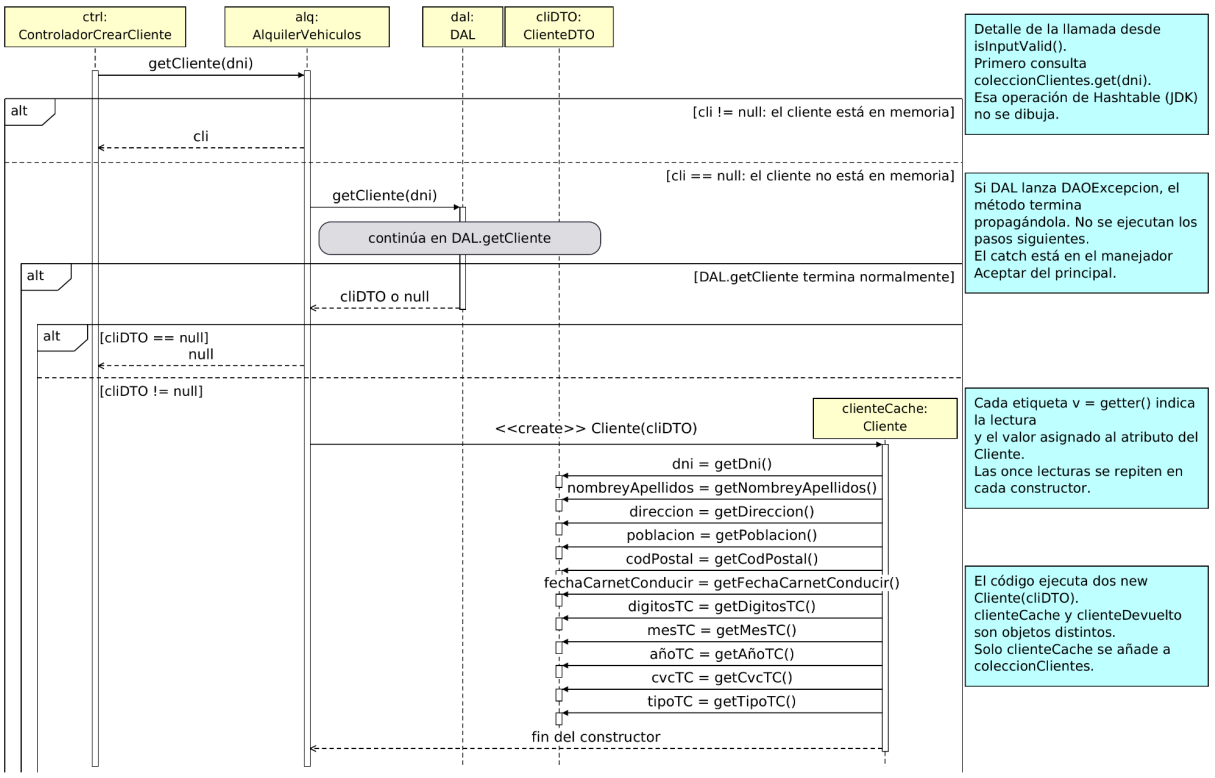
- CVC: longitud de 3 caracteres y convertible a entero.

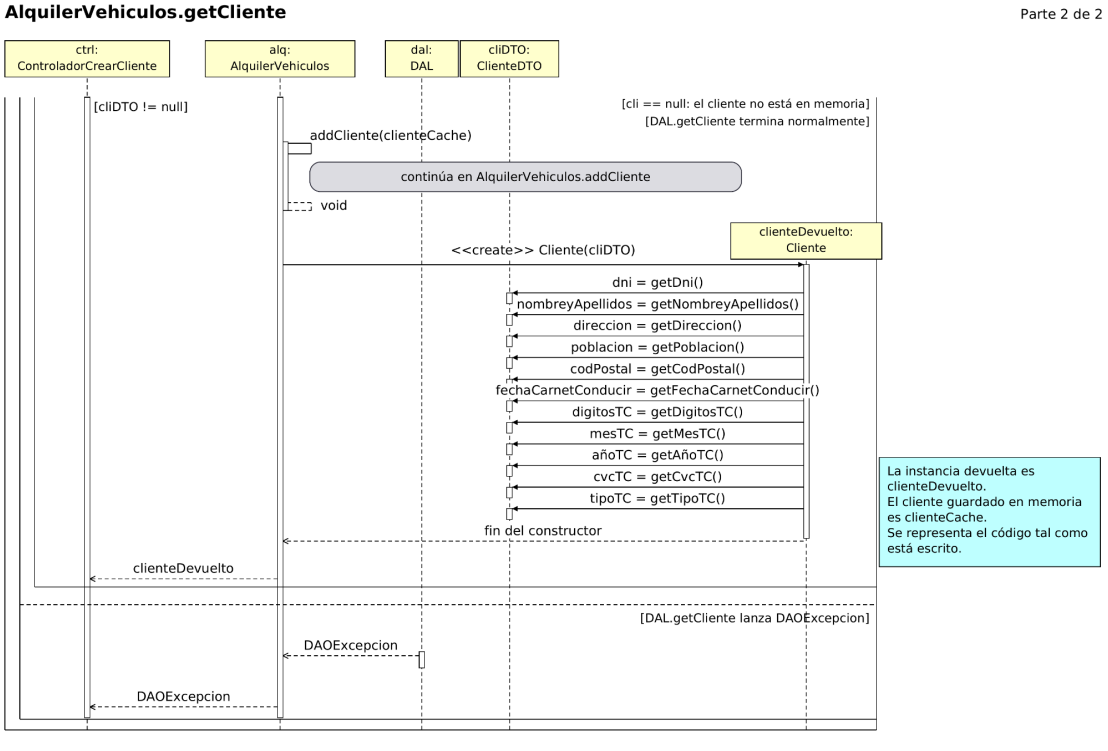
- Tipo de tarjeta: no vacío y con longitud máxima de 10 caracteres.

Las llamadas a esInt del código postal y del CVC son condicionales debido a la evaluación en cortocircuito del operador ||. Los métodos auxiliares esInt y esLong devuelven false cuando capturan una NumberFormatException.

Si errorMessage queda vacío, el método devuelve true. En caso contrario, crea una alerta de tipo WARNING, establece su título y contenido, la muestra mediante showAndWait() y devuelve false. Las posibles DAOExcepcion se propagan hasta el manejador del botón Aceptar.

3. AlquilerVehiculos.getCliente





Este método busca primero el DNI en la colección de clientes mantenida en memoria. Si encuentra un objeto Cliente, lo devuelve directamente, sin consultar la capa de persistencia.

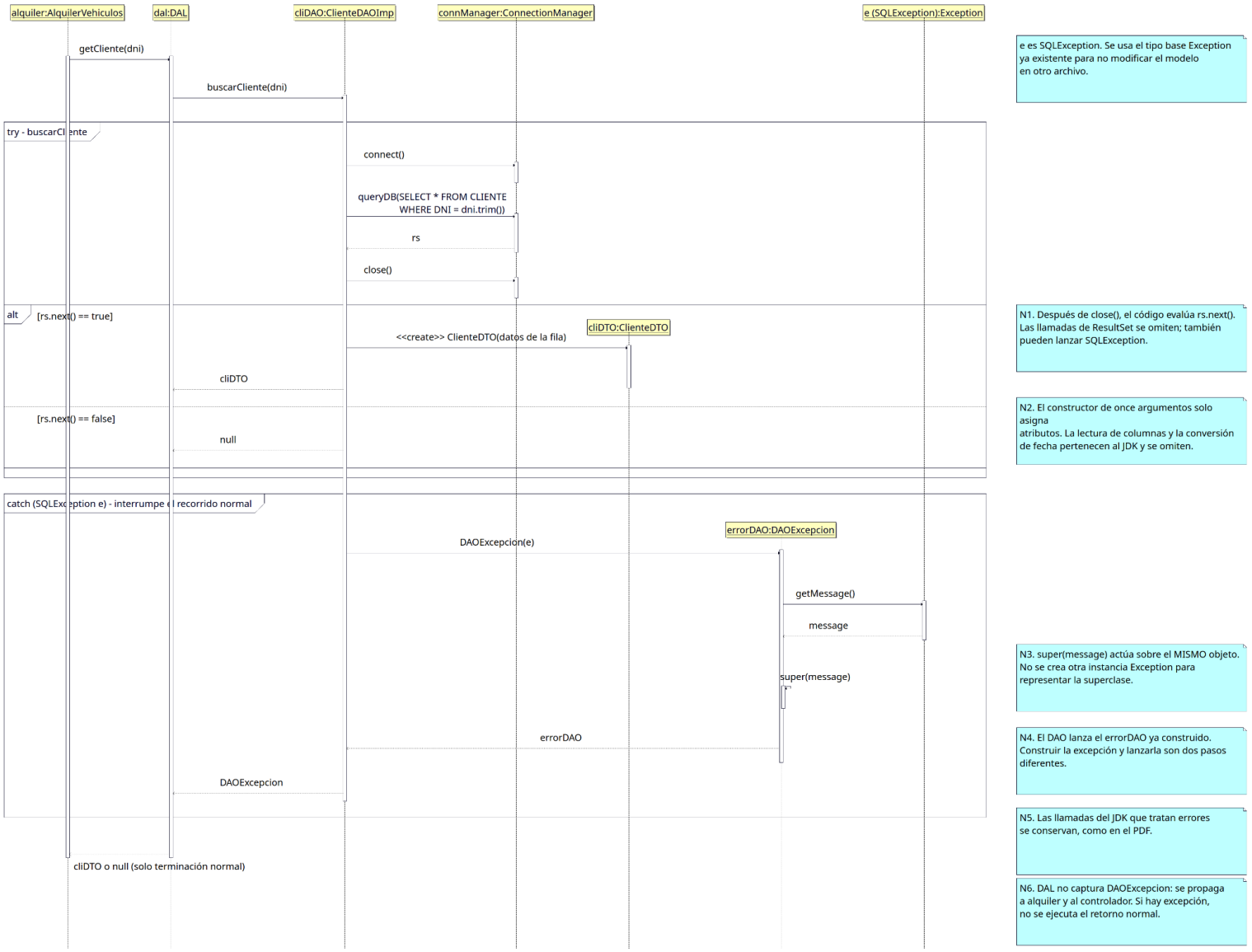
Si no lo encuentra, invoca DAL.getCliente(dni). Cuando DAL devuelve null, el método también devuelve null, indicando que la búsqueda no ha encontrado al cliente.

Si DAL devuelve un ClienteDTO, se construye un objeto Cliente, identificado en el diagrama como clienteCache, y se incorpora a la colección mediante addCliente(...). Después se construye un segundo objeto Cliente a partir del mismo DTO, identificado como clienteDevuelto, que es el que se devuelve al controlador.

Se representan dos objetos distintos porque el código contiene dos expresiones new Cliente(cliDTO). Cada constructor obtiene los once datos del DTO mediante sus getters. Si DAL lanza una DAOExcepcion, esta se propaga y no se ejecutan las construcciones posteriores.

(Para la creación de este diagrama de secuencia se ha utilizado edición con IA a la hora de su ilustración debido a problemas de visibilidad, como la desaparición de algunas líneas verticales).

4. DAL.getCliente



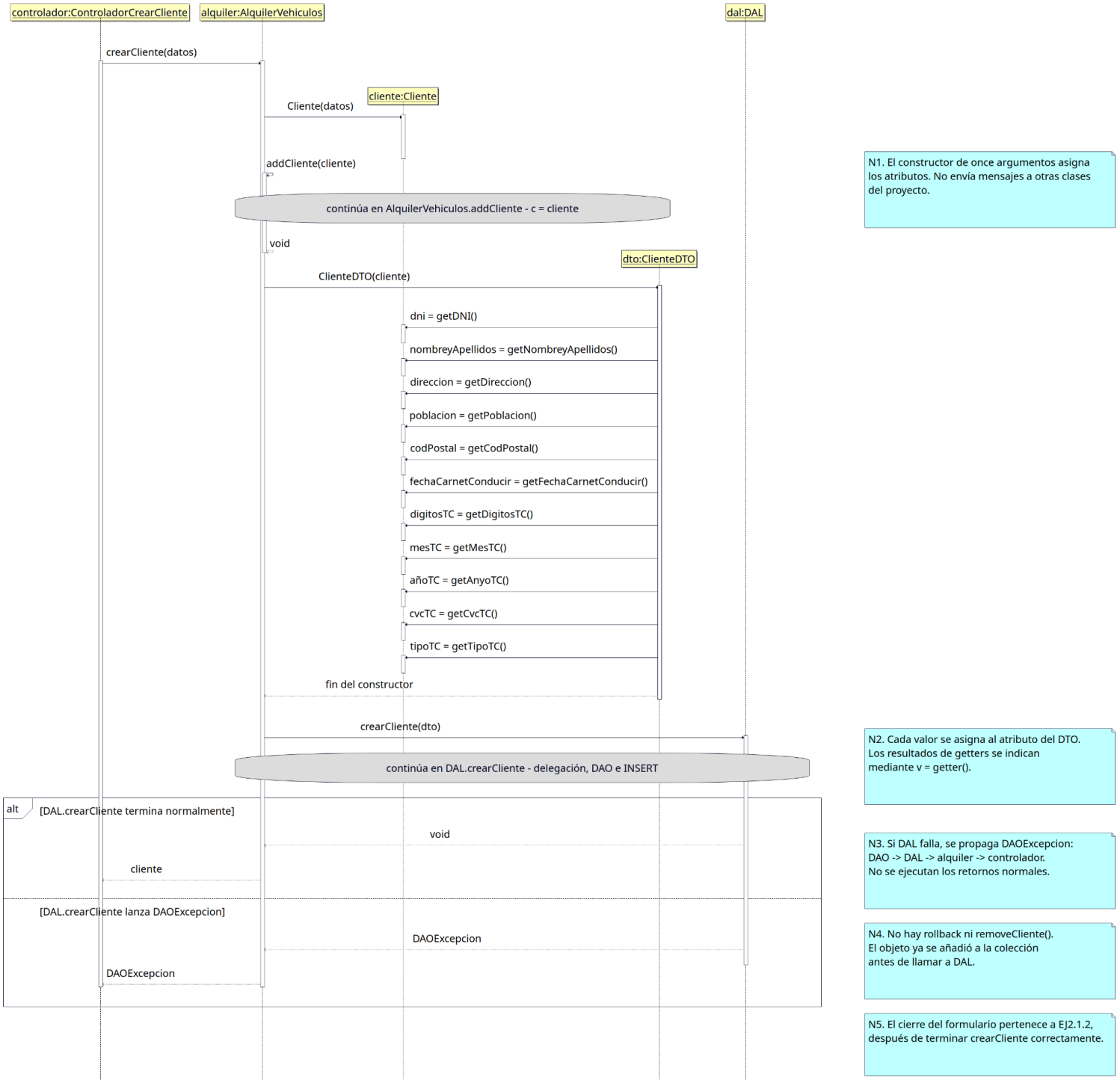
DAL delega la búsqueda en buscarCliente(dni) del objeto ClienteDAOImp.

El DAO solicita a ConnectionManager que establezca la conexión, ejecuta una consulta sobre la tabla CLIENTE utilizando el DNI sin espacios en sus extremos y llama a close(). Después evalúa si el resultado contiene una fila mediante rs.next(), conservando el orden escrito en el código.

Si existe una fila, obtiene sus datos y construye un ClienteDTO, que devuelve a DAL. Si no existe, devuelve null. DAL transmite el resultado a AlquilerVehiculos.

Si se produce una SQLException dentro del bloque protegido, el DAO construye una DAOExcepcion y la lanza. Su constructor obtiene el mensaje de la excepción original y lo pasa al constructor de la superclase mediante super(e.getMessage()). DAL no captura esta excepción, por lo que continúa propagándose hacia el controlador.

5. AlquilerVehiculos.crearCliente

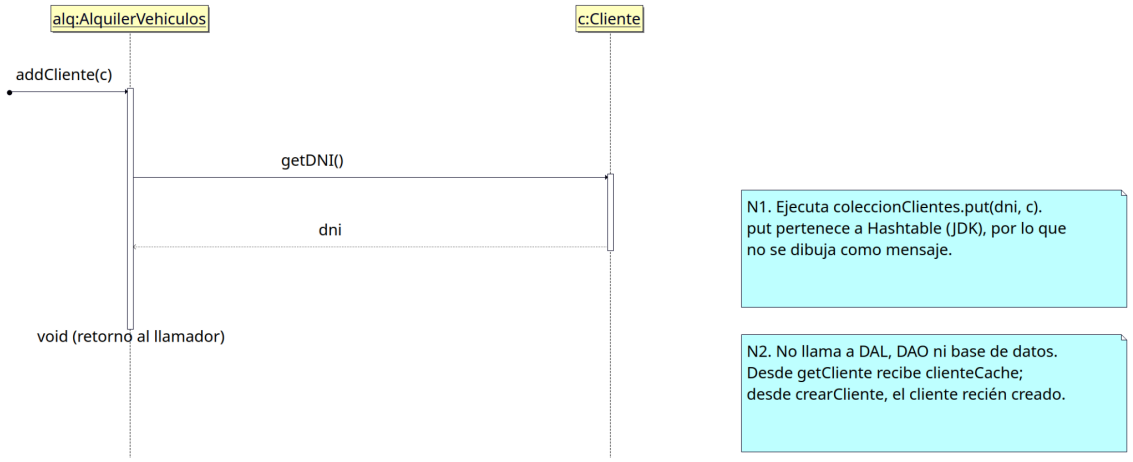


El método recibe los once datos del formulario y construye un objeto Cliente. A continuación, invoca addCliente(cliente) para incorporarlo a la colección en memoria.

Después construye un ClienteDTO a partir del cliente creado. Durante esta construcción, el DTO consulta los getters del objeto Cliente y copia sus valores. Finalmente, llama a DAL.crearCliente(dto) para solicitar la persistencia y, si la operación termina correctamente, devuelve el cliente.

El orden es significativo: la incorporación a memoria se realiza antes de la llamada a persistencia. Si esta última lanza una DAOExcepcion, el método la propaga y no devuelve normalmente el cliente. El código no elimina de la colección el objeto previamente incorporado.

6. AlquilerVehiculos.addCliente

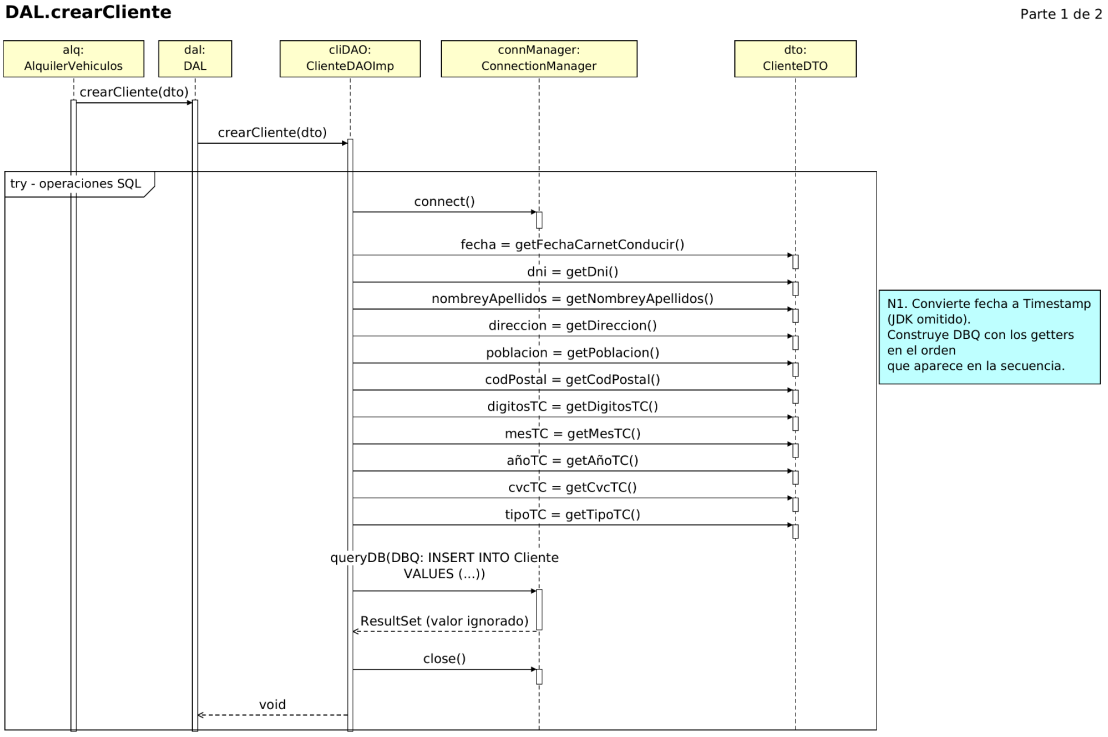


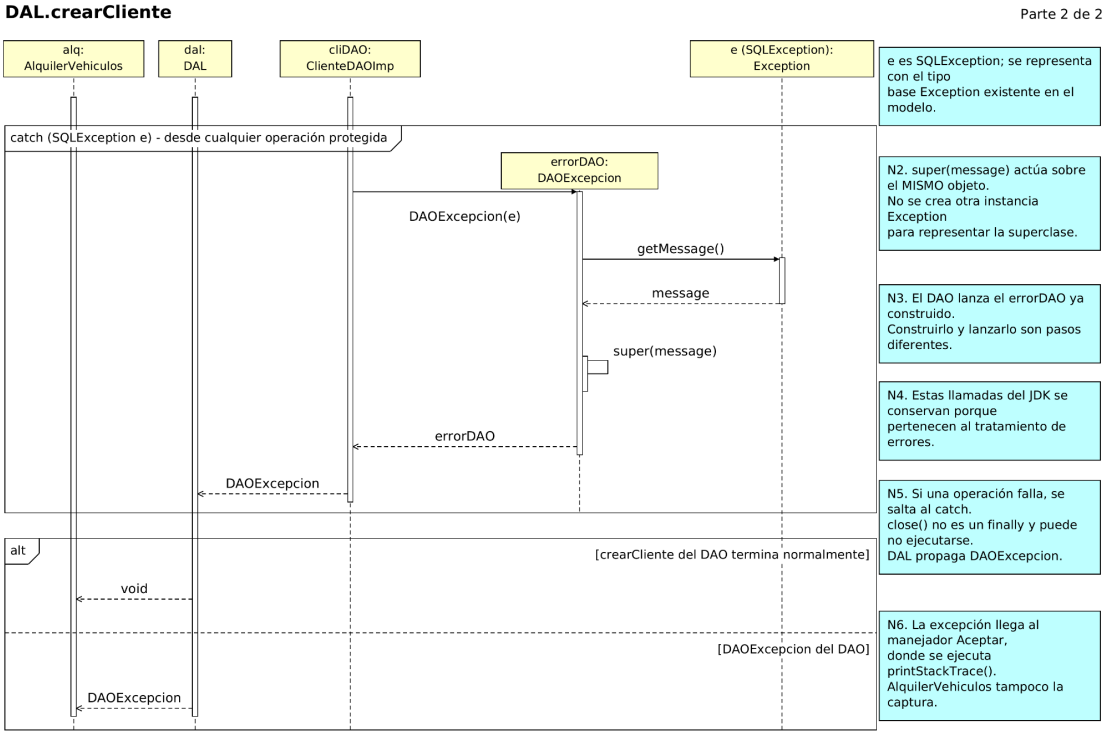
Este método recibe un objeto Cliente, obtiene su DNI mediante getDNI() y lo utiliza como clave para almacenarlo en coleccionClientes.

Su responsabilidad se limita a actualizar la colección en memoria. No invoca DAL ni realiza una inserción en la base de datos. La operación put(...) de Hashtable se explica mediante una nota al pertenecer a la biblioteca de Java.

El método se utiliza tanto para incorporar un cliente recuperado de persistencia como para añadir uno creado durante el alta.

7. DAL.crearCliente





DAL recibe el ClienteDTO y delega la operación en ClienteDAOImp.crearCliente(dto).

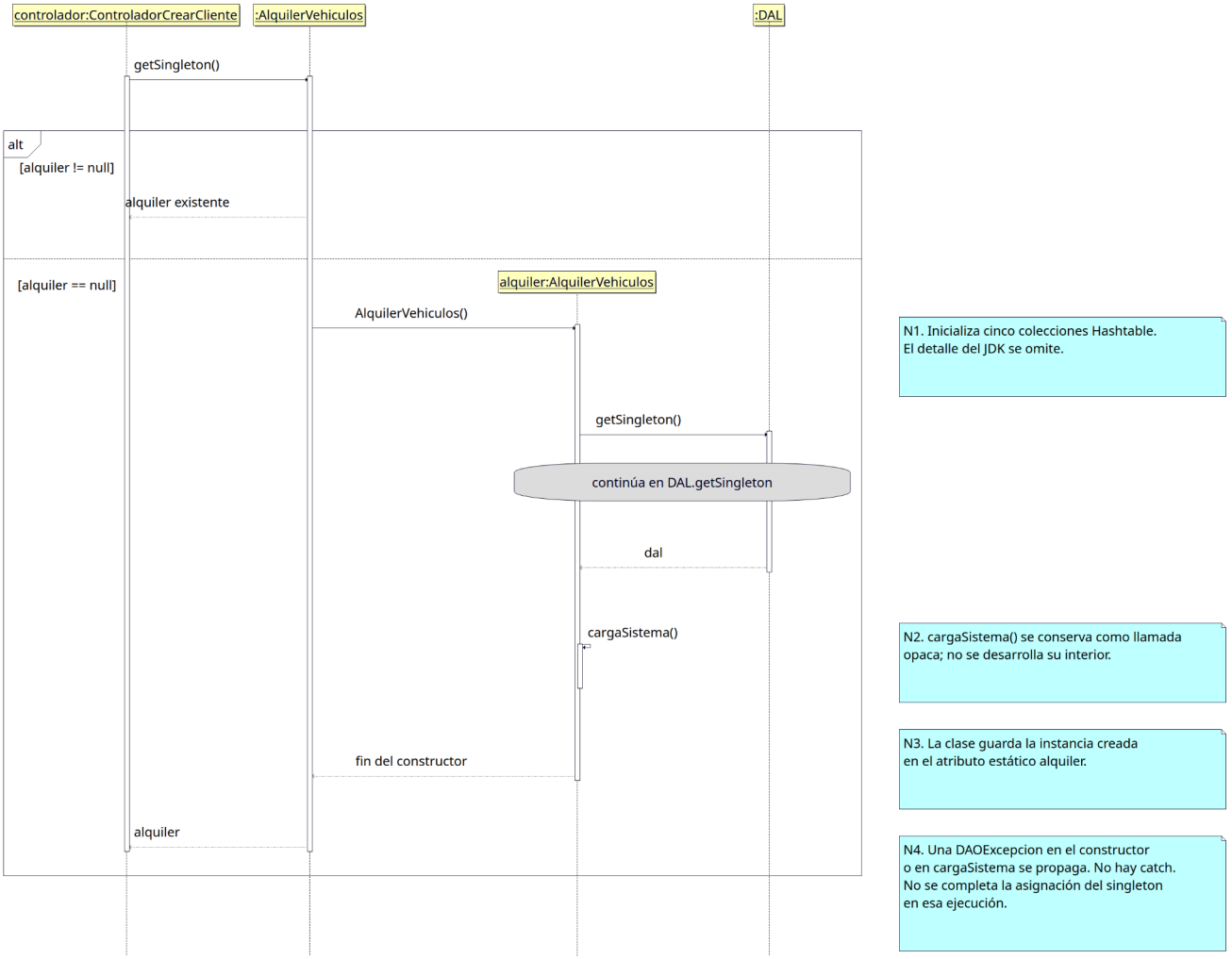
El DAO establece la conexión, obtiene la fecha de expedición del carnet y la convierte a Timestamp. Después consulta los demás getters del DTO y construye la sentencia SQL de inserción. La envía mediante ConnectionManager.queryDB(DBQ) y, si la ejecución continúa normalmente, cierra la conexión.

Se representa queryDB(...) porque es el método utilizado por la implementación existente para enviar el INSERT. Su valor de retorno no se utiliza.

Si se produce una SQLException, el DAO construye y lanza una DAOExcepcion. DAL y AlquilerVehiculos la propagan hasta el manejador de Aceptar. El cierre de la conexión no está situado en un bloque finally, por lo que puede no ejecutarse si falla una operación anterior.

(Al igual que pasó con AlquilarVehiculo.getCliente, se ha utilizado edición de IA en la imagen para ajustarla mejor por problemas visuales con la imagen original a la hora de la exportación)

8. AlquilerVehiculos.getSingleton

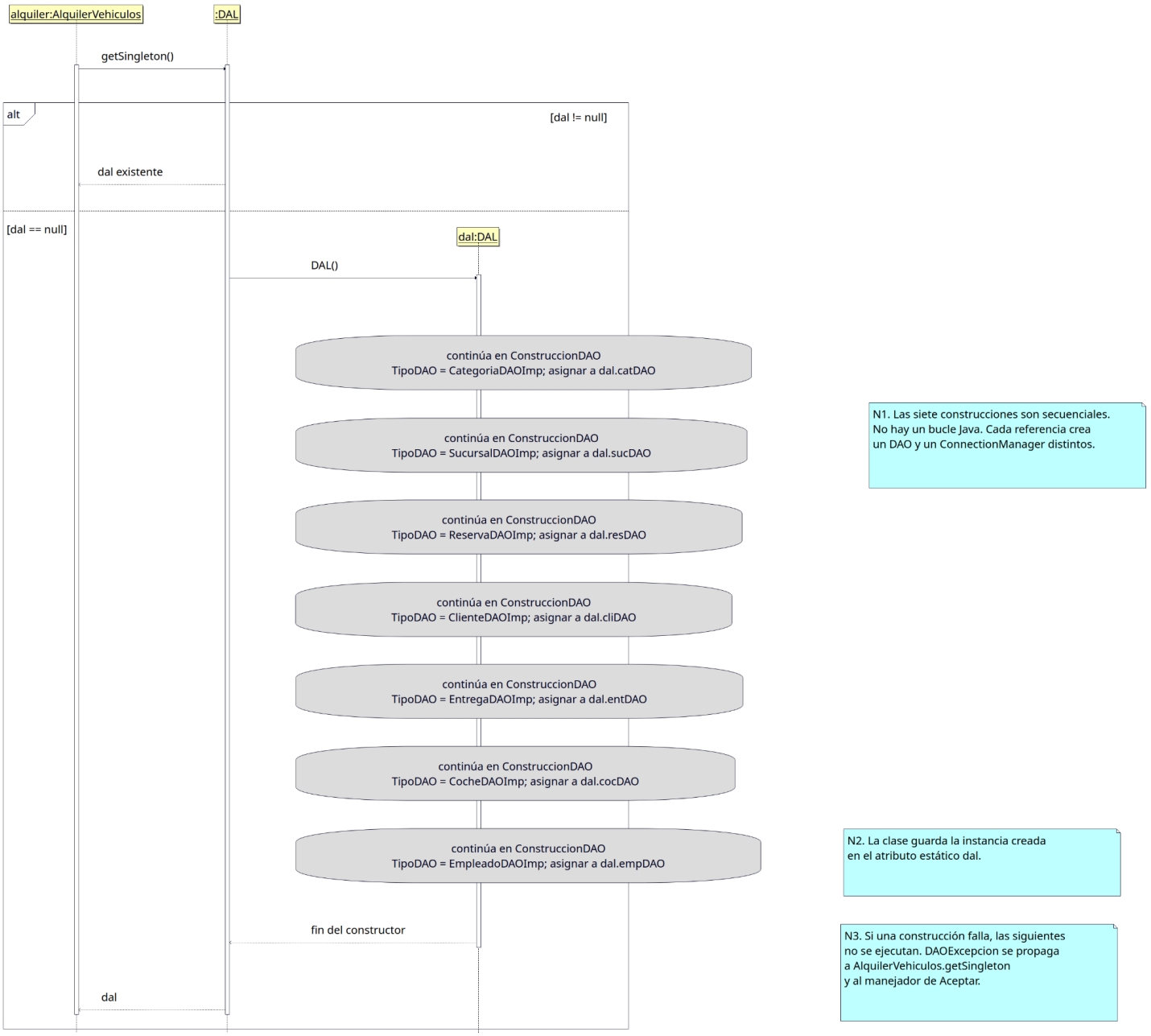


Este método proporciona la instancia compartida de AlquilerVehiculos. Si el atributo estático alquiler ya contiene una instancia, la devuelve directamente.

Si es nulo, ejecuta el constructor de AlquilerVehiculos. Este inicializa cinco colecciones, obtiene la instancia de DAL mediante DAL.getSingleton() y llama a cargaSistema(). El interior de esta última operación no se desarrolla, conforme a las indicaciones del ejercicio.

Cuando la construcción termina correctamente, la instancia se asigna al atributo estático y se devuelve. Si se produce una DAOExcepcion, se propaga y no se completa esa asignación.

9. DAL.getSingleton



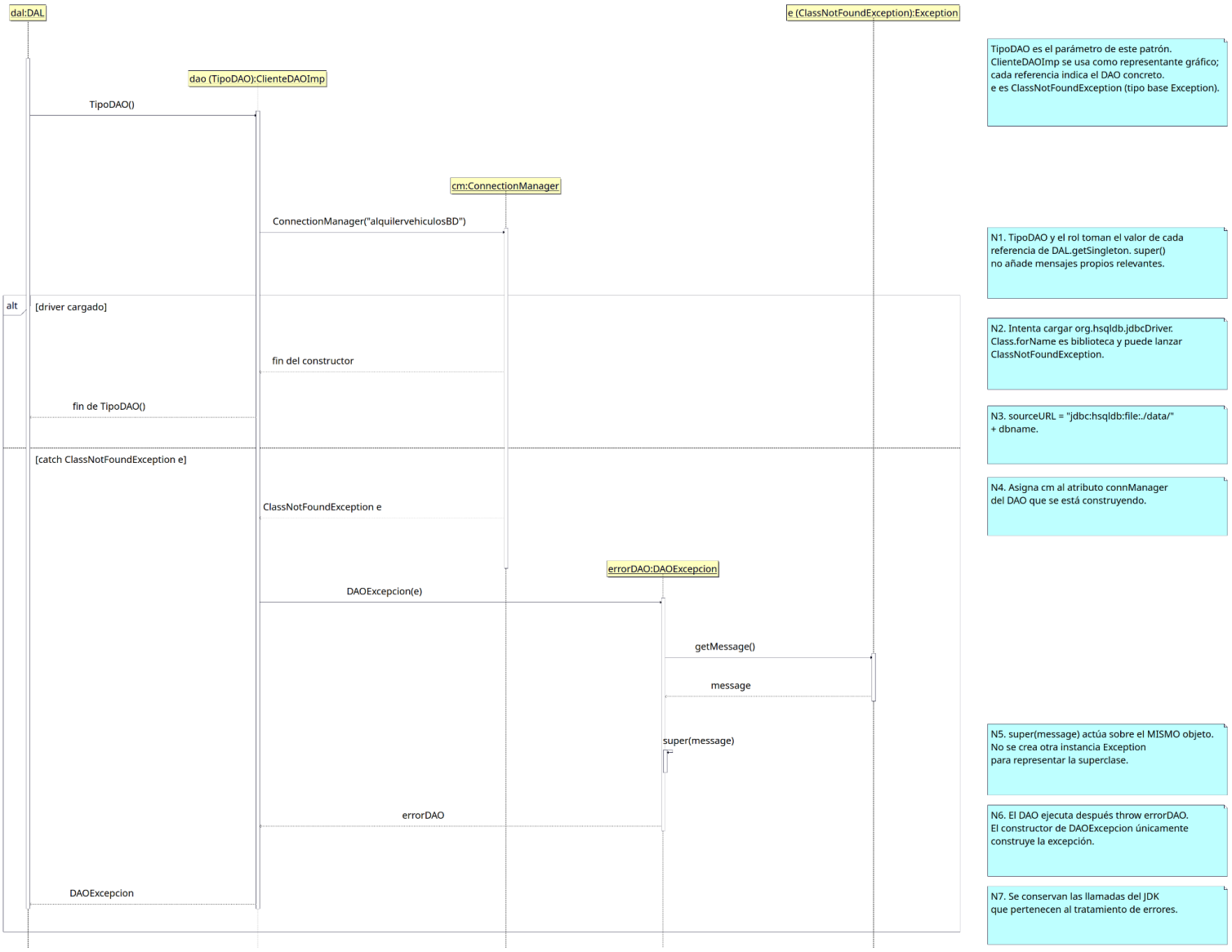
Este método devuelve la instancia compartida de DAL si ya existe. En caso contrario, ejecuta su constructor.

El constructor crea, en este orden, los objetos CategoriaDAOImp, SucursalDAOImp, ReservaDAOImp, ClienteDAOImp, EntregaDAOImp, CocheDAOImp y

EmpleadoDAOImp. Son siete construcciones secuenciales, no un bucle ni operaciones paralelas.

Cada construcción se desarrolla mediante la continuación ConstruccionDAO, indicando el tipo concreto y el atributo al que se asigna el objeto. Si alguna falla, las siguientes no se ejecutan y la excepción se propaga. Cuando el constructor termina correctamente, se guarda y devuelve la instancia de DAL.

10. ConstruccionDAO



Este diagrama recoge el patrón común de construcción de los siete DAO. TipoDAO representa el tipo concreto indicado en cada continuación y no una clase adicional del programa.

Cada DAO construye su propio ConnectionManager, pasando el nombre de la base de datos. El constructor de este gestor intenta cargar el controlador de HSQLDB y prepara la URL de conexión. Si termina correctamente, el gestor se asigna al atributo connManager del DAO.

Si la carga del controlador provoca una ClassNotFoundException, el DAO la captura, construye una DAOExcepcion y la lanza. La construcción de la excepción y su lanzamiento son pasos distintos. La excepción se propaga por los métodos de inicialización hasta el manejador de Aceptar, donde se imprime su traza.

2.2 Entregar vehículo reservado

La entrega empieza en el menú principal de la aplicación. El empleado elige una sucursal, una reserva que esté pendiente y un vehículo de la categoría correspondiente e introduce los datos de la entrega. La operación registra la entrega y actualiza el estado del coche.

2.2.1 Diagrama de clases

El diagrama del EJ2.2.1 que se corresponde con el segundo caso de uso contiene de la vista del EJ2.2 29 clases e interfaces. Para identificarlas se siguió desde el manejador de la interfaz hasta las operaciones de persistencia y se incluyeron los DTO, objetos de dominio y la excepción que interviene.

Organización del modelo

Presentación. ControladorPrincipal es quien abre el caso de uso de la entrega de un vehículo reservado. ControladorEntregarVehiculoReserva hereda de ControladorCasoDeUso, que se encarga de la inicialización común. El controlador específico gestiona las tablas y el formulario de entrega y trabaja con los objetos de Reserva y Coche.

Dominio y lógica. AlquilerVehiculos reúne las operaciones del caso de uso. Cada reserva está asociada a un cliente, una categoría y las sucursales de recogida y devolución. La Entrega relaciona la reserva con el coche que se entrega y el empleado que realiza la operación. Si no hay coches disponibles en la categoría solicitada, se puede recorrer la jerarquía de Categoria para buscar en las categorías superiores.

Acceso a datos. AlquilerVehiculos accede a los datos a través de DAL, que trabaja con interfaces DAO. Las implementacionese de estas interfaces usan ConnectionManager para realizar las consultas y escrituras en la base de datos. Las capas intercambian datos mediante DTO y los errores de persistencia se representan con DAOExcepcion.

Alcance y representación

El diagrama general se divide en cuatro diagramas/vistas parciales, desde el EJ2.2.1a hasta el EJ2.2.1d, para que sea más fácil su lectura. Algunas clases aparecen en varias vistas para que se puedan seguir las relaciones entre las capas. Tanto el diagrama general como las cuatro vistas parciales se han exportado directamente desde BoUML. Los diagramas editables se encuentran en A7.EJ2y3.

Devolucion y Danyo quedan fuera del caso de uso de la entrega. Categoria y Sucursal sí intervienen como objetos de dominio. Sus DAO y DTO se utilizan durante la carga del sistema, un proceso que queda fuera del alcance del ejercicio. Los tipos de JavaFX y del JDK forman parte del entorno de ejecución.

Vista general del diagrama de clases

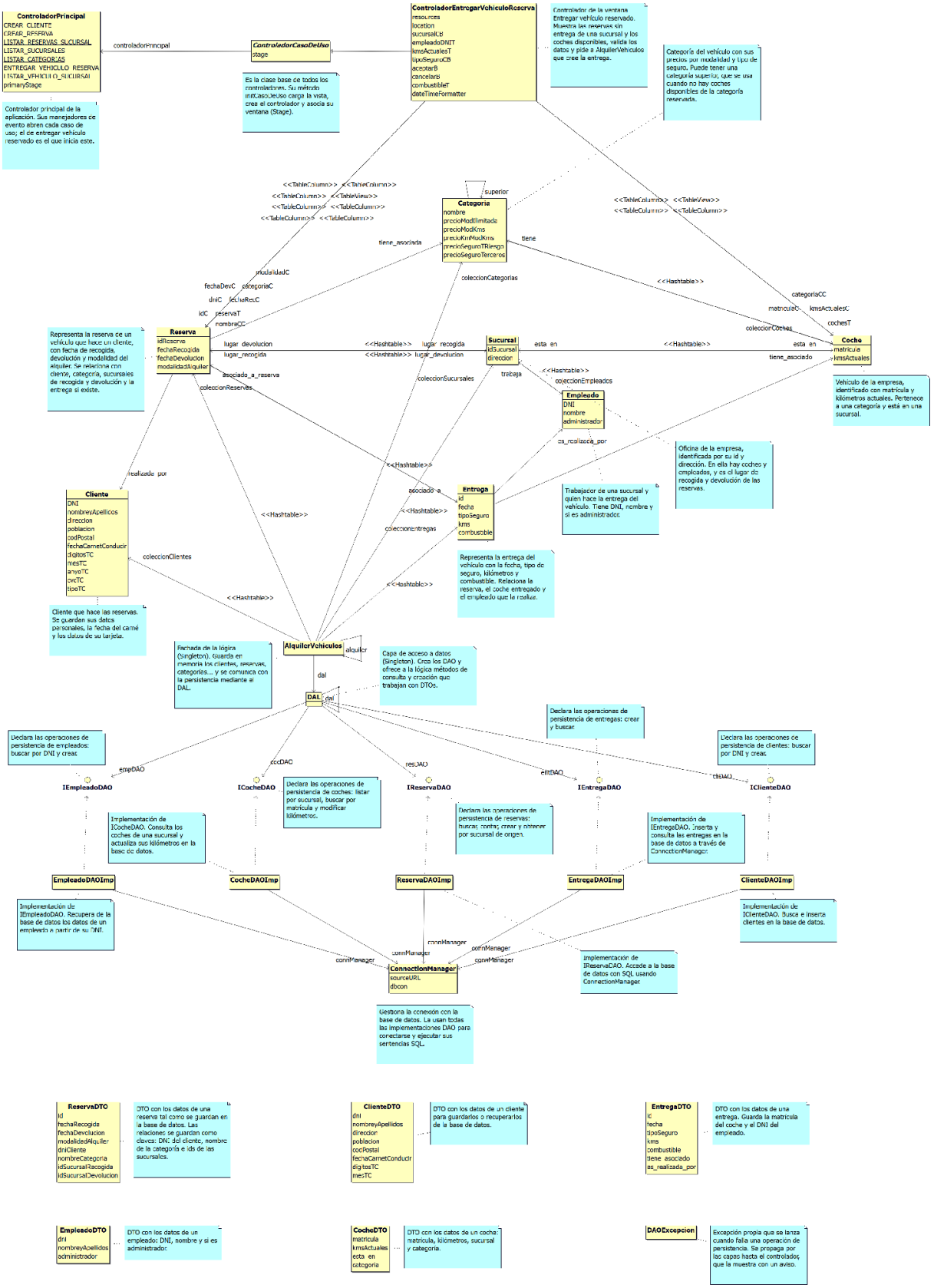


Figura 1. Vista general del diagrama de clases EJ2.2.1.

Clases de presentación

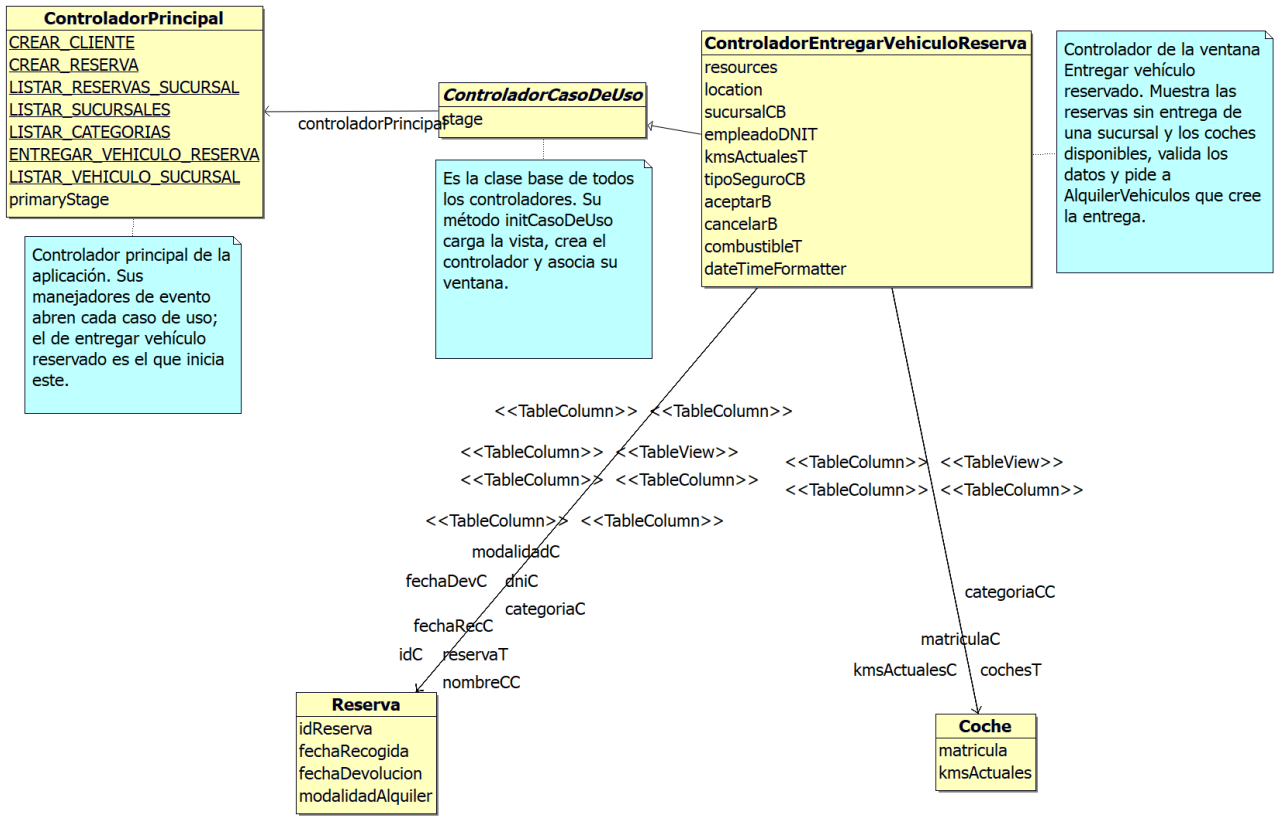


Figura 2. Vista de presentación EJ2.2.1a.

Elemento	Responsabilidad en el caso de uso
ControladorPrincipal	Cuando el usuario selecciona la opción de entrega en el menú, se ejecuta entregarVehiculoReserva. Este método solicita la inicialización del caso de uso y muestra su ventana.
ControladorCasoDeUso	Es la clase abstracta que contiene todos los pasos comunes para abrir un caso de uso. Carga el FXML, obtiene el controlador de la vista y le pasa la ventana y el controlador principal.
ControladorEntregarVehiculoReserva	Prepara el formulario y carga las reservas y los coches. Antes de registrar la entrega, valida los datos del formulario y, si son correctos, le pide a AlquilerVehiculos que la cree.
AlquilerVehiculos	Actúa como fachada de la lógica de negocio. Gestiona las colecciones de objetos del dominio y coordina las consultas y escrituras en la base de datos a través del DAL.

Relaciones del dominio

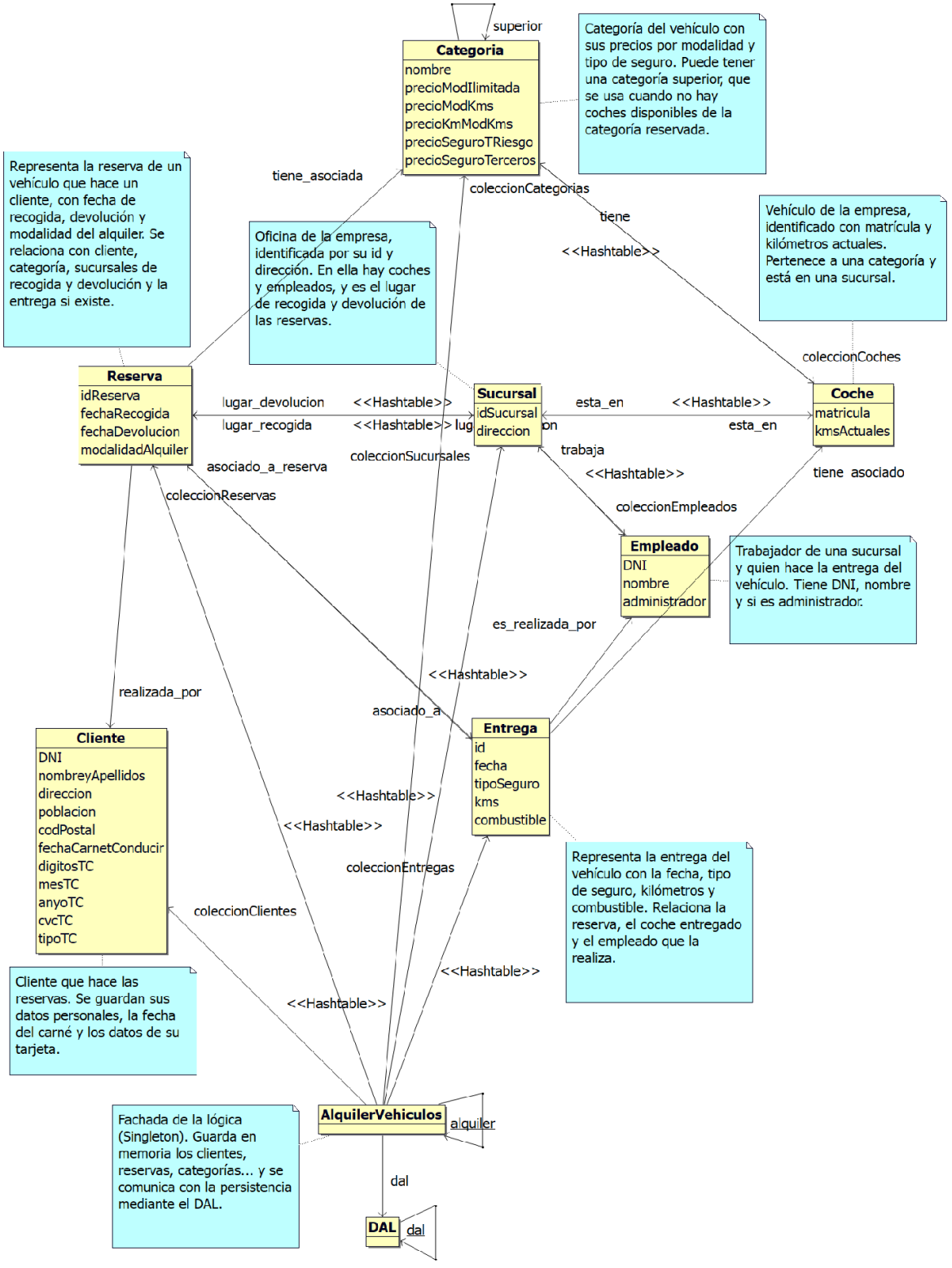


Figura 3. Vista del dominio EJ2.2.1b.

Objetos del dominio

Elemento	Responsabilidad en el caso de uso
Reserva	Representa la solicitud de alquiler. Guarda las fechas y la modalidad del alquiler y relaciona la reserva con el cliente, la categoría y las sucursales de recogida y devolución. Si ya se ha entregado el vehículo, también referencia la entrega.
Entrega	Guarda la fecha, tipo de seguro, kilometraje y nivel de combustible en el momento de entregar el coche. También relaciona la reserva con el coche entregado y el empleado que realizó la entrega.
Coche	Se identifica por su matrícula y guarda el kilometraje. Pertenece a una categoría y puede estar asignado a una sucursal.
Categoria	Agrupa los coches que sean del mismo tipo y establece sus tarifas. Si no hay coches disponibles en la categoría solicitada, su referencia a la categoría superior permite continuar la búsqueda.
Sucursal	Representa una oficina de alquiler y guarda su identificador y dirección. Puede ser el punto de recogida o devolución de una reserva y tener coches y empleados asignados.
Empleado	Representa al trabajador que realiza la entrega. El formulario usa su DNI para comprobar que existe y asociarlo a la entrega.
Cliente	Representa a quien hizo la reserva. Su nombre y DNI se muestran en la tabla de reservas de la sucursal.

Clases e interfaces de persistencia

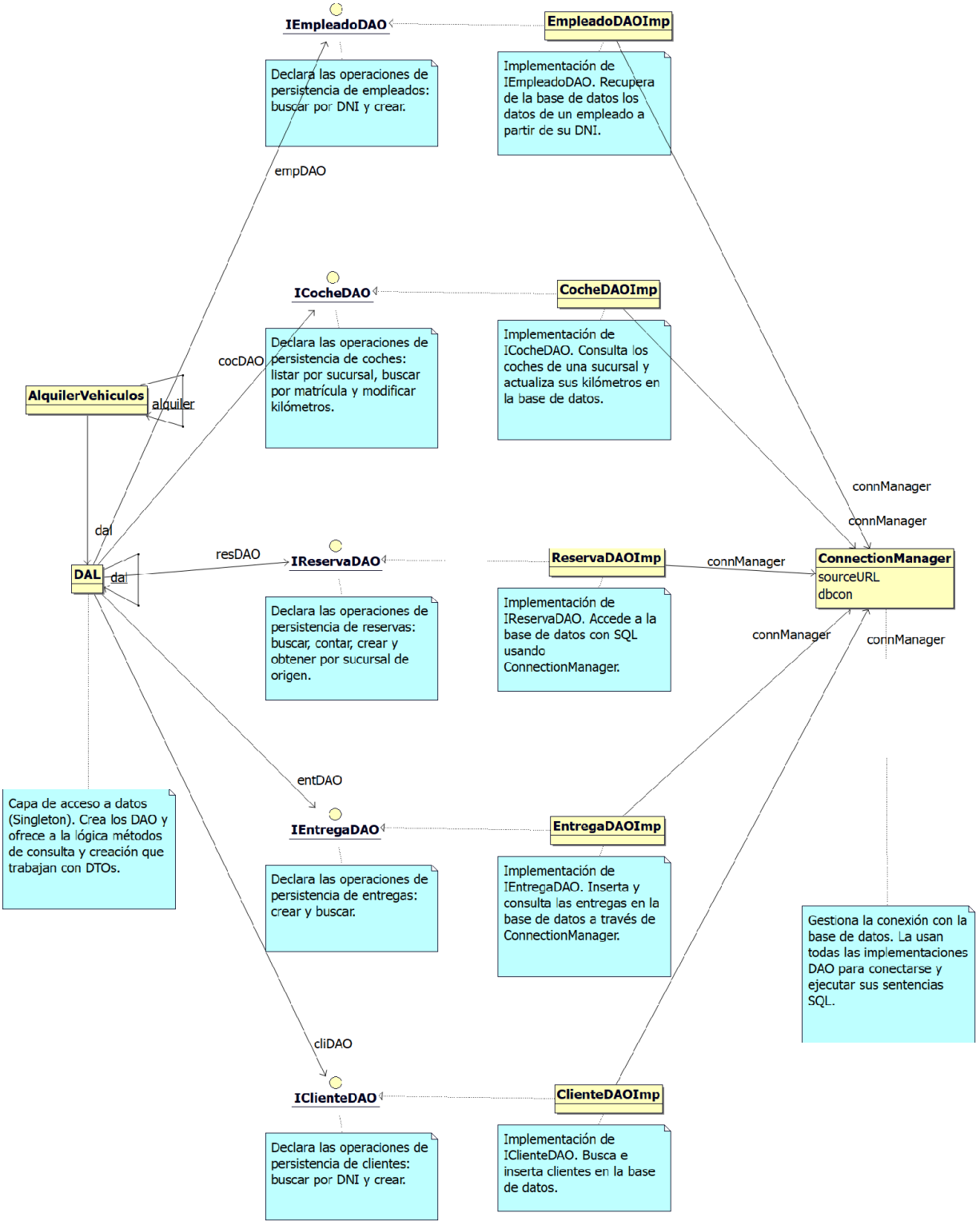


Figura 4. Vista de acceso a datos EJ2.2.1c.

Responsabilidades de persistencia

Elemento	Responsabilidad en el caso de uso
DAL	Sirve de punto de entrada a la capa de acceso a datos. Delega cada operación en el DAO correspondiente y devuelve a la lógica los datos de las consultas mediante DTO.
IReservaDAO	Define las operaciones de acceso a los datos de las reservas. En este caso se usa para consultar las reservas de una sucursal de recogida o buscar una reserva por su identificador.
ReservaDAOImp	Implementa IReservaDAO. Ejecuta las consultas y convierte los registros obtenidos en ReservaDTO.
IEntregaDAO	Define las operaciones para guardar una entrega y consultarla por su identificador.
EntregaDAOImp	Implementa IEntregaDAO. Guarda la entrega en la base de datos y si la encuentra al consultarla, devuelve un EntregaDTO con sus datos.
ICocheDAO	Define las operaciones para listar los coches de una sucursal, buscar un coche por su matrícula y actualizar sus datos.
CocheDAOImp	Implementa ICocheDAO. Consulta los coches en la base de datos y actualiza la sucursal y el kilometraje del vehículo al registrar la entrega.
IEmpleadoDAO	Define las operaciones de acceso a los datos de los empleados. En este caso se usa para buscar al empleado por su DNI.
EmpleadoDAOImp	Implementa IEmpleadoDAO. Busca en la base de datos al empleado cuyo DNI se ha indicado.
IClienteDAO	Define las operaciones de acceso a los datos de los clientes. En este caso se usa para buscar al cliente por su DNI.
ClienteDAOImp	Implementa IClienteDAO. Recupera de la base de datos al cliente asociado a una reserva cuando la lógica necesita sus datos.
ConnectionManager	Abre y cierra la conexión con la base de datos y ejecuta las sentencias SQL que le pasan las implementaciones de los DAO.

Datos intercambiados y excepción

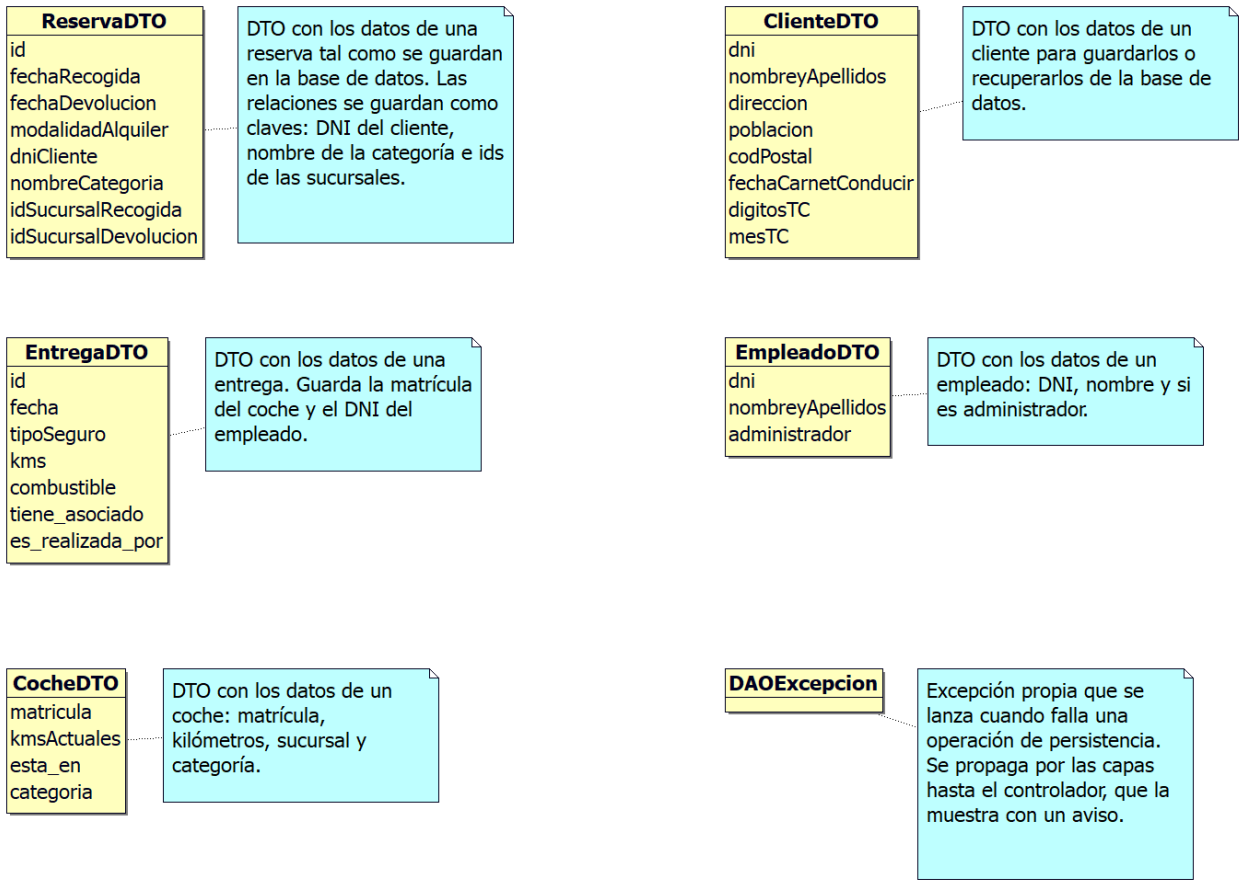


Figura 5. Vista de DTO y excepción EJ2.2.1d.

Responsabilidades de los DTO y la excepción

Elemento	Responsabilidad en el caso de uso
ReservaDTO	Transfiere los datos de una reserva almacenada. Identifica al cliente por su DNI e incluye la categoría y los identificadores de las sucursales.
EntregaDTO	Transfiere los datos de una entrega al consultarla o registrarla en la base de datos. Incluye el identificador de la reserva, la matrícula del coche y el DNI del empleado.
CocheDTO	Transfiere matrícula, kilometraje, sucursal y categoría del coche al listar o recuperar vehículos.
EmpleadoDTO	Contiene el DNI, nombre y si el empleado es administrador. Estos datos se usan para construir el objeto Empleado.
ClienteDTO	Contiene los datos personales y de la tarjeta del cliente. Se usa para reconstruir el objeto Cliente asociado a una reserva.
DAOExcepcion	Representa los errores que se producen al acceder a los datos. La forma de tratar el error depende de en qué parte del caso de uso se capture la excepción.

2.2.2 Diagrama de secuencia

El diagrama EJ2.2.2 comienza cuando el usuario selecciona la opción de entrega de un vehículo reservado en el menú principal. Esta acción ejecuta el manejador entregarVehiculoReserva(event). Las 23 continuaciones de la vista del EJ2.2 muestran con más detalle las llamadas entre las diferentes capas de presentación, lógica y persistencia. En cada diagrama, los eventos que llegan desde fuera se marcan con un punto de entrada.

Apertura y carga de las reservas

ControladorPrincipal llama a initCasoDeUso para abrir la ventan. El controlador base carga el FXML y durante la carga, se crea ControladorEntregarVehiculoReserva y se ejecuta initialize. Este método obtiene las sucursales, ordena sus identificadores, carga los seguros «todoriesgo» y «terceros» y registra los escuchadores. Al seleccionar la primera sucursal se ejecuta popularTablaReservas. El mismo método vuelve a ejecutarse cada vez que el empleado cambia de sucursal.

popularTablaReservas vacía la tabla y llama a listarReservasSucursal para obtener las reservas de la sucursal seleccionada. El DAL delega la consulta en ReservaDAOImp. A partir de cada ReservaDTO, AlquilerVehciulos reconstruye el objeto Reserva, le asigna las referencias al cliente, las sucursales, la categoría y la entrega, si existe y lo guarda en memoria. La tabla solo muestra las reservas cuyo método getAsociado_a() devuelve null, es decir, las que todavía no tienen una entrega asociada.

Selección de un vehículo

Cuando el empleado selecciona una reserva, listarCoches obtiene la categoría solicitada y consulta los coches de la sucursal seleccionada. La lógica construye los objetos Coche a partir de sus DTO y filtra por categoría. Si no encuentra ningún coche, repite la búsqueda en la categoría inmediatamente superior. El proceso continúa hasta que encuentra coches o ya no quedan categorías superiores. Si se agotan las opciones, el controlador muestra un aviso.

Validación y registro de la entrega

Al pulsar Aceptar, isInputValid comprueba que se hayan seleccionado una reserva, coche y seguro, que el kilometraje y el combustible pueden convertirse a double y que el DNI corresponde a un empleado existente. Si hay varios errores de validación, los junta en un único aviso.

Si los datos son váldios, aceptar llama a crearEntrega con la fecha y horas actuales, el identificador de la reserva, los datos del formulario, la matrícula y el DNI del empleado. crearEntrega prepara el EntregaDTO e inserta el registro en la base de datos. Después ejecuta dos sentencias UPDATE sobre el coche. Una deja la sucursal a NULL y la otra guarda el kilometraje indicado. Por último, crea el objeto Entrega, lo vincula con la reserva, el empleado y el coche, y lo añade a la colección de entregas. Cuando la operación termina correctamente, el controlador cierra la ventan.

Tratamiento de errores

En las operaciones DAO que aparecen en el diagrama, las excepciones SQLException se convierten en DAOExcepcion. Si initialize, popularTablaReservas o listarCoches capturan

esa excepción, muestran un aviso. En los caminos de error representados para aceptar y el constructor de Reserva, se imprime la traza de la excepción.

El diagrama también muestra la llamada a cargaSistema() al inicializar el singleton. El funcionamiento interno de este método también queda fuera del alcance del ejercicio.

Secuencia principal y criterio de lectura

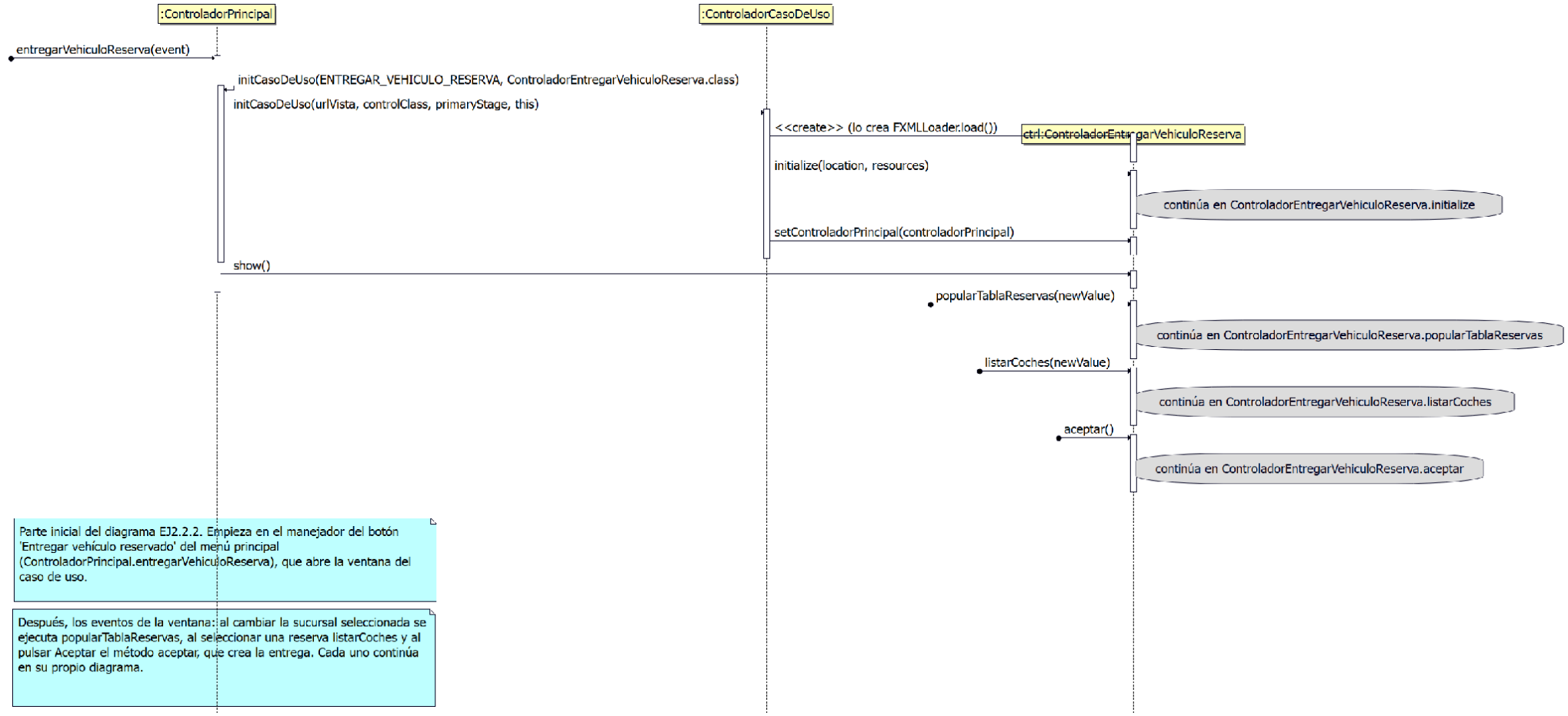


Figura 6. EJ2.2.2. Apertura de la ventana y eventos del caso de uso.

El orden de los mensajes se sigue de arriba abajo. Cada flecha indica quién envía el mensaje y quién lo recibe. Los rectángulos verticales sobre las líneas de vida muestran los periodos de activación. Cuando una llamada se desarrolla en otra figura, el rótulo de continuación indica qué método se detalla allí.

Las continuaciones se agrupan según la clase que inicia cada secuencia. En las llamadas de acceso a datos se muestra la implementación concreta del DAO que atiende la llamada en tiempo de ejecución. Las consultas sencillas a colecciones en memoria se describen dentro del método que las realiza. Las operaciones de biblioteca se muestran de forma resumida y se incluyen los avisos y las excepciones que afectan al flujo del caso de uso.

Continuaciones de presentación

initialize

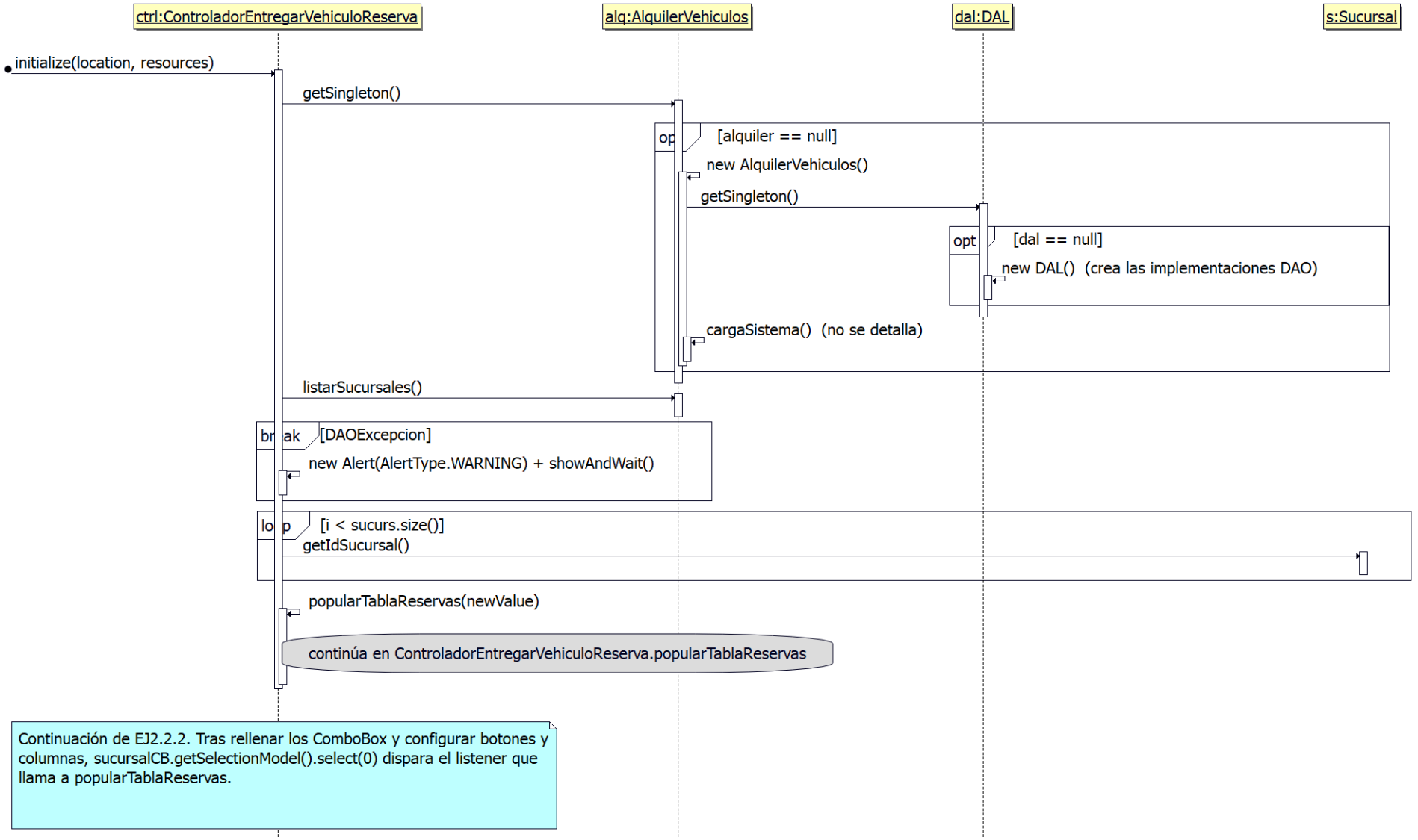


Figura 7. ControladorEntregarVehiculoReserva.initialize.

popularTablaReservas

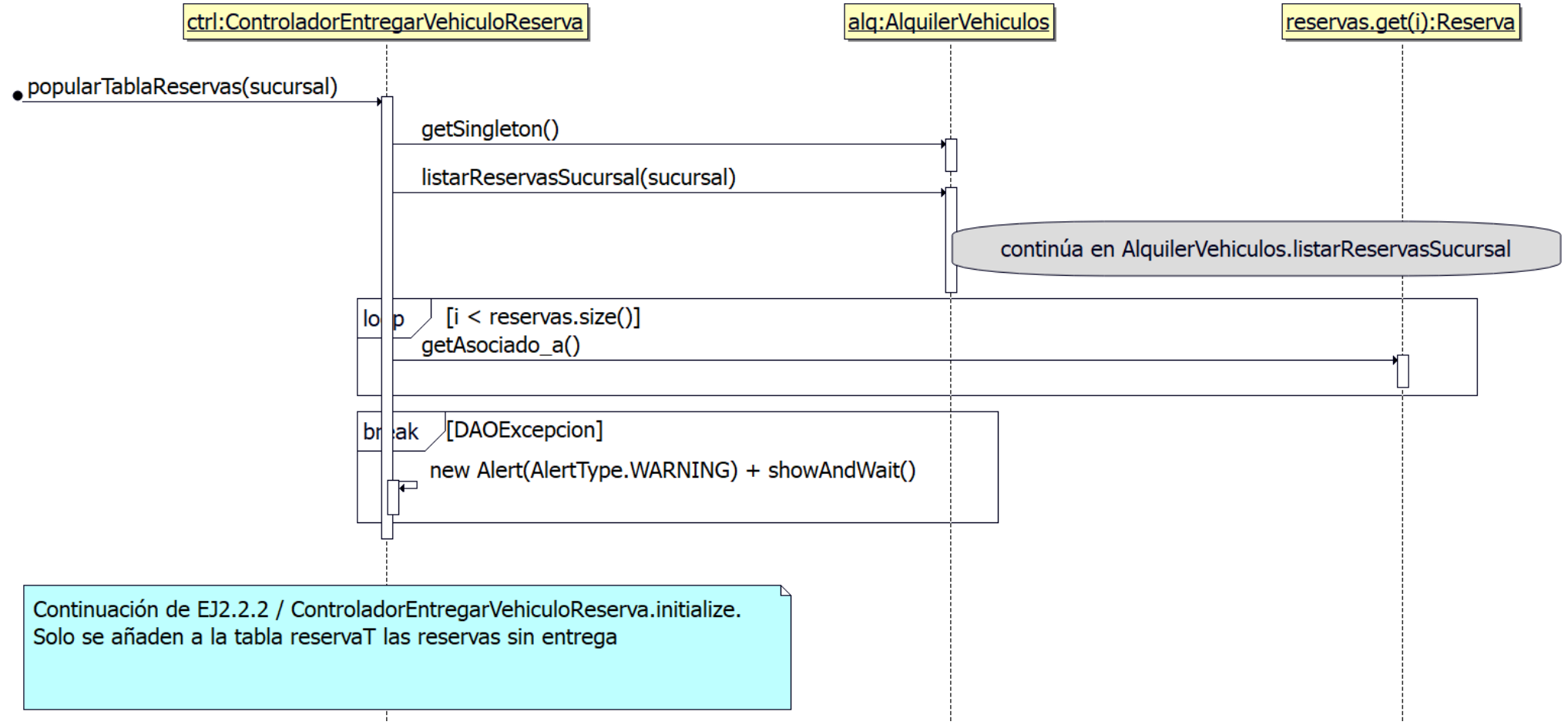


Figura 8. ControladorEntregarVehiculoReserva.popularTablaReservas.

listarCoches

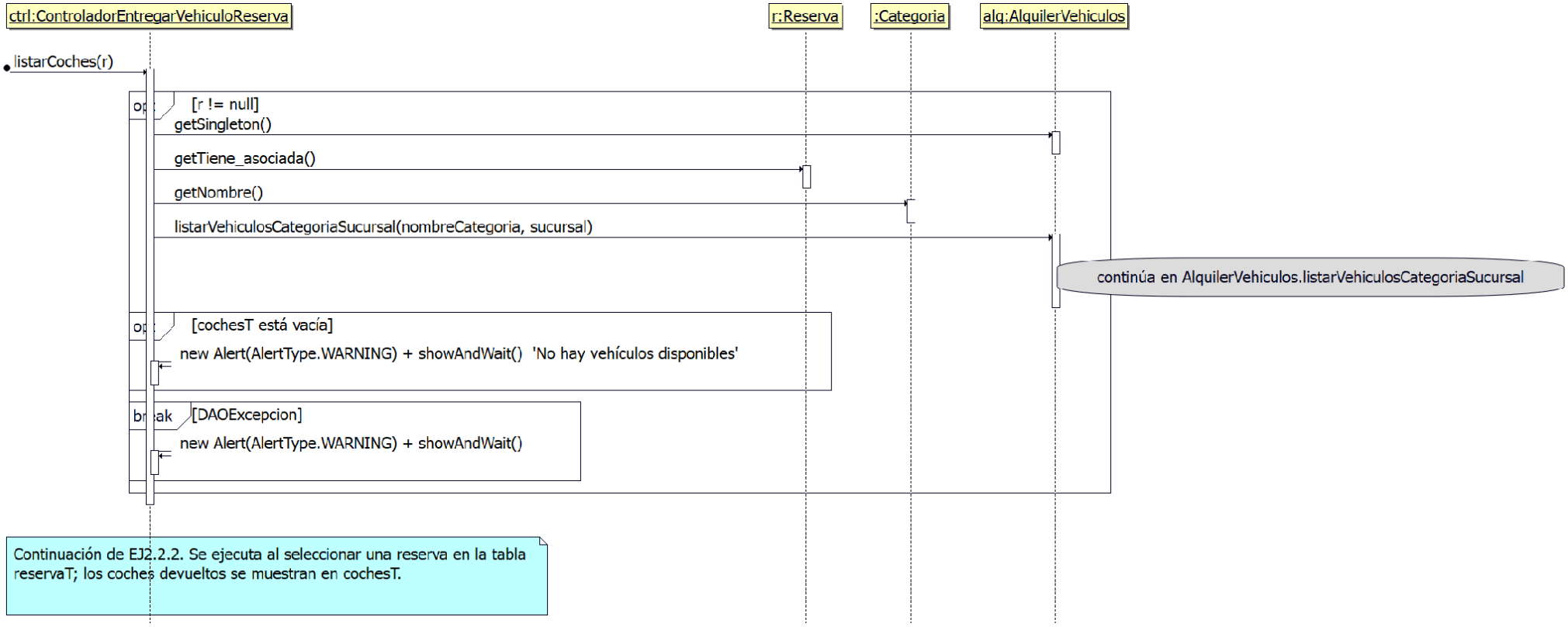


Figura 9. ControladorEntregarVehiculoReserva.listarCoches.

aceptar

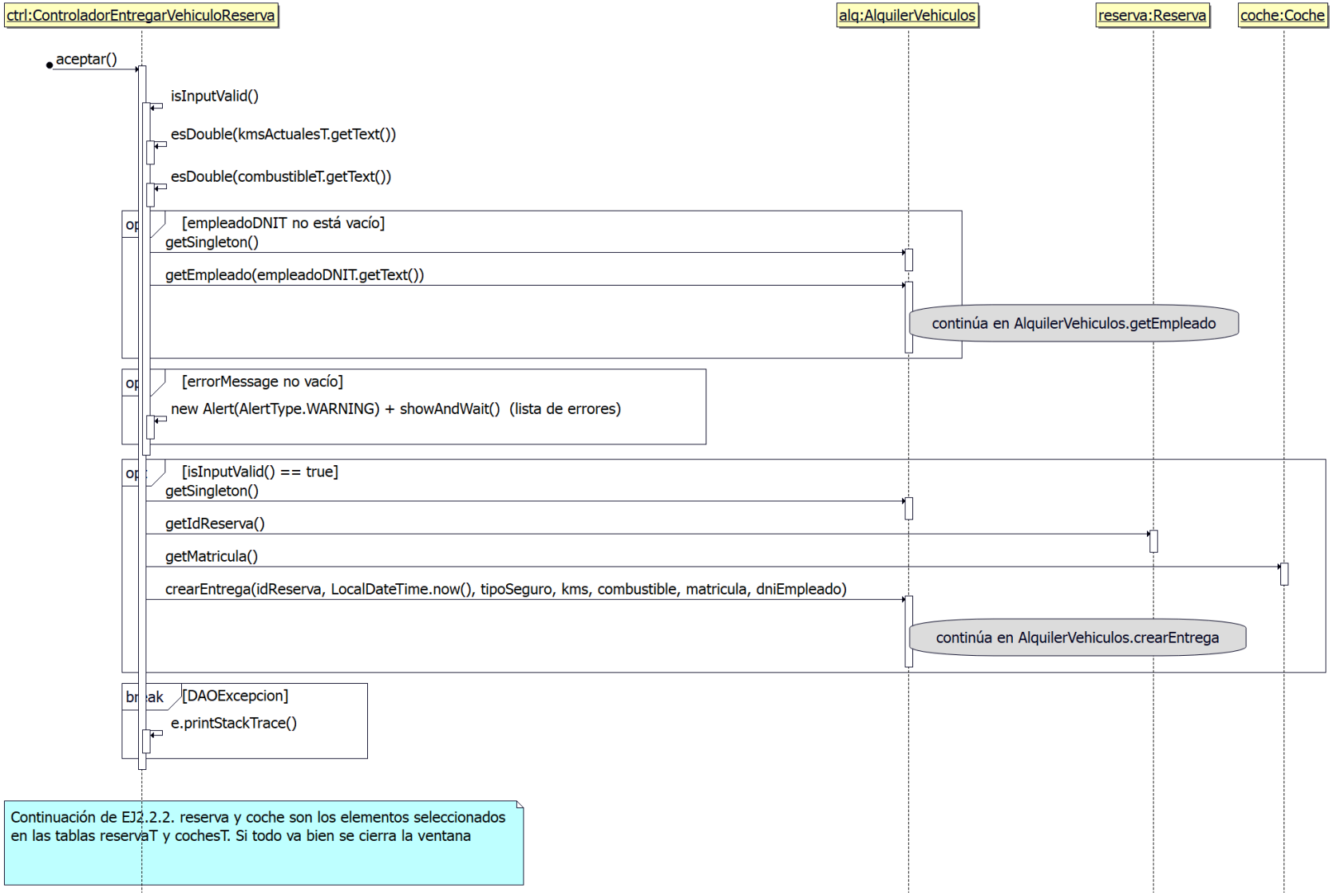


Figura 10. ControladorEntregarVehiculoReserva.aceptar.

Continuaciones de lógica

AlquilerVehiculos.listarReservasSucursal

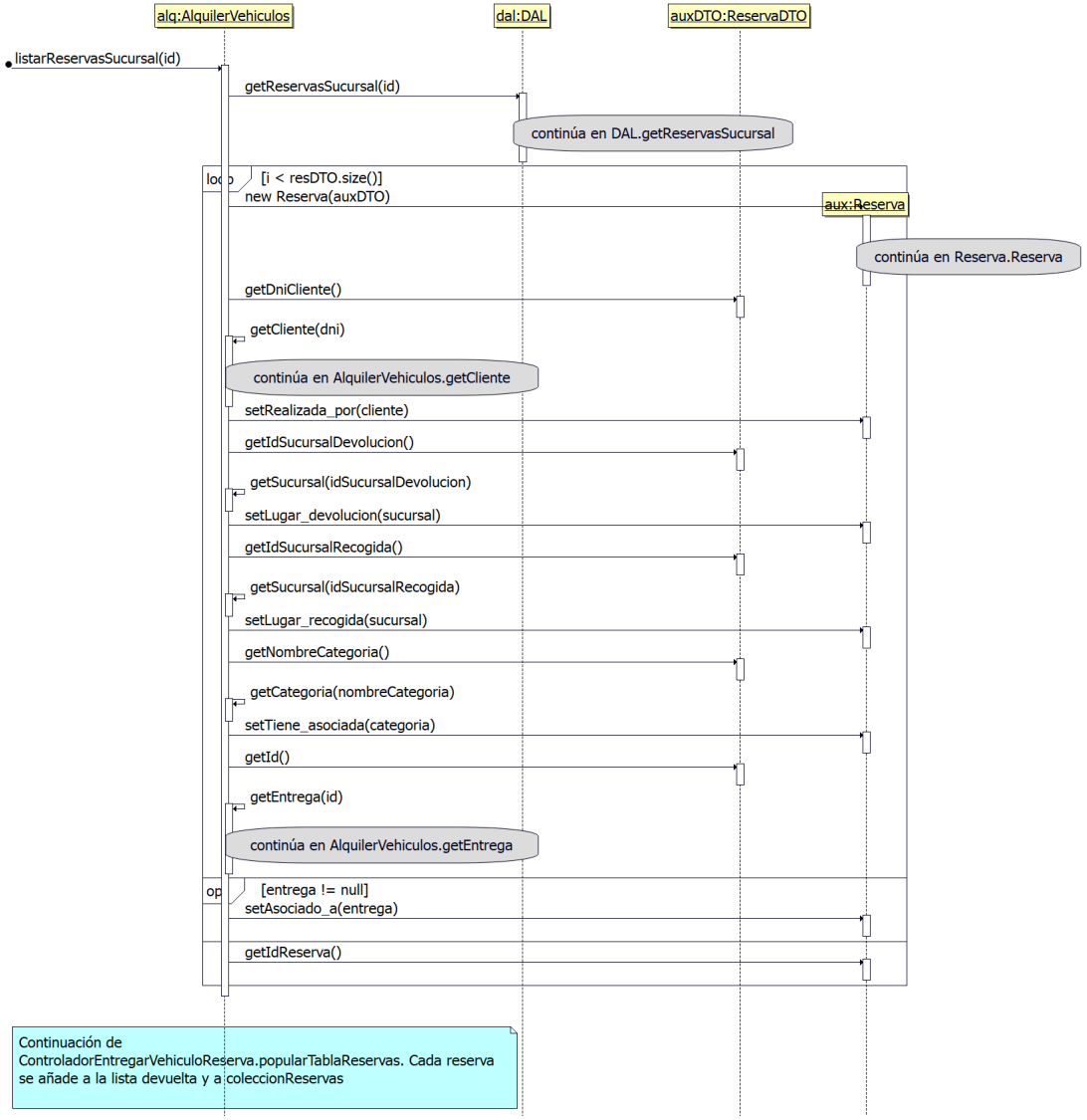


Figura 11. AlquilerVehiculos.listarReservasSucursal.

AlquilerVehiculos.getCliente

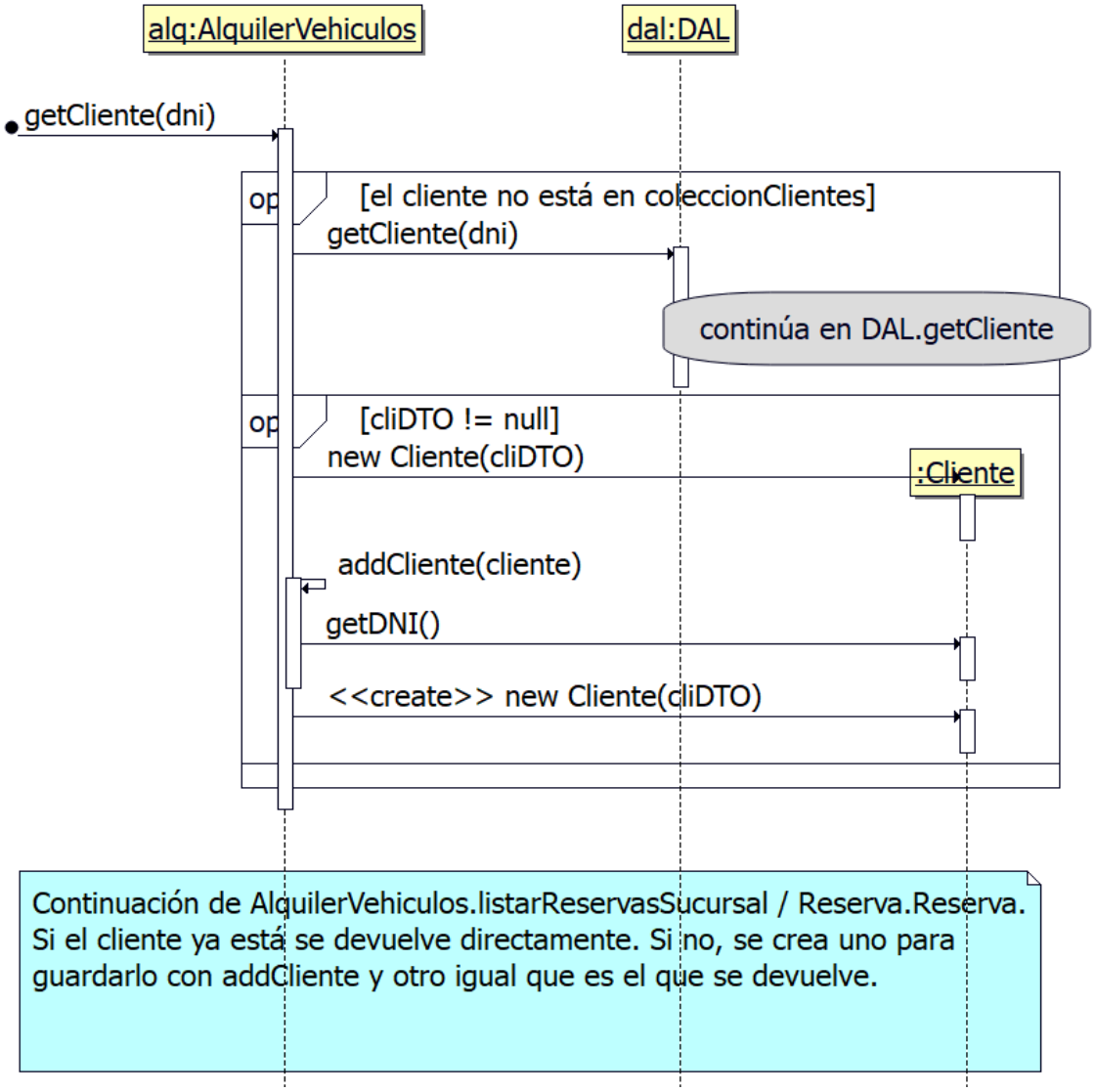


Figura 12. AlquilerVehiculos.getCliente.

AlquilerVehiculos.getEntrega

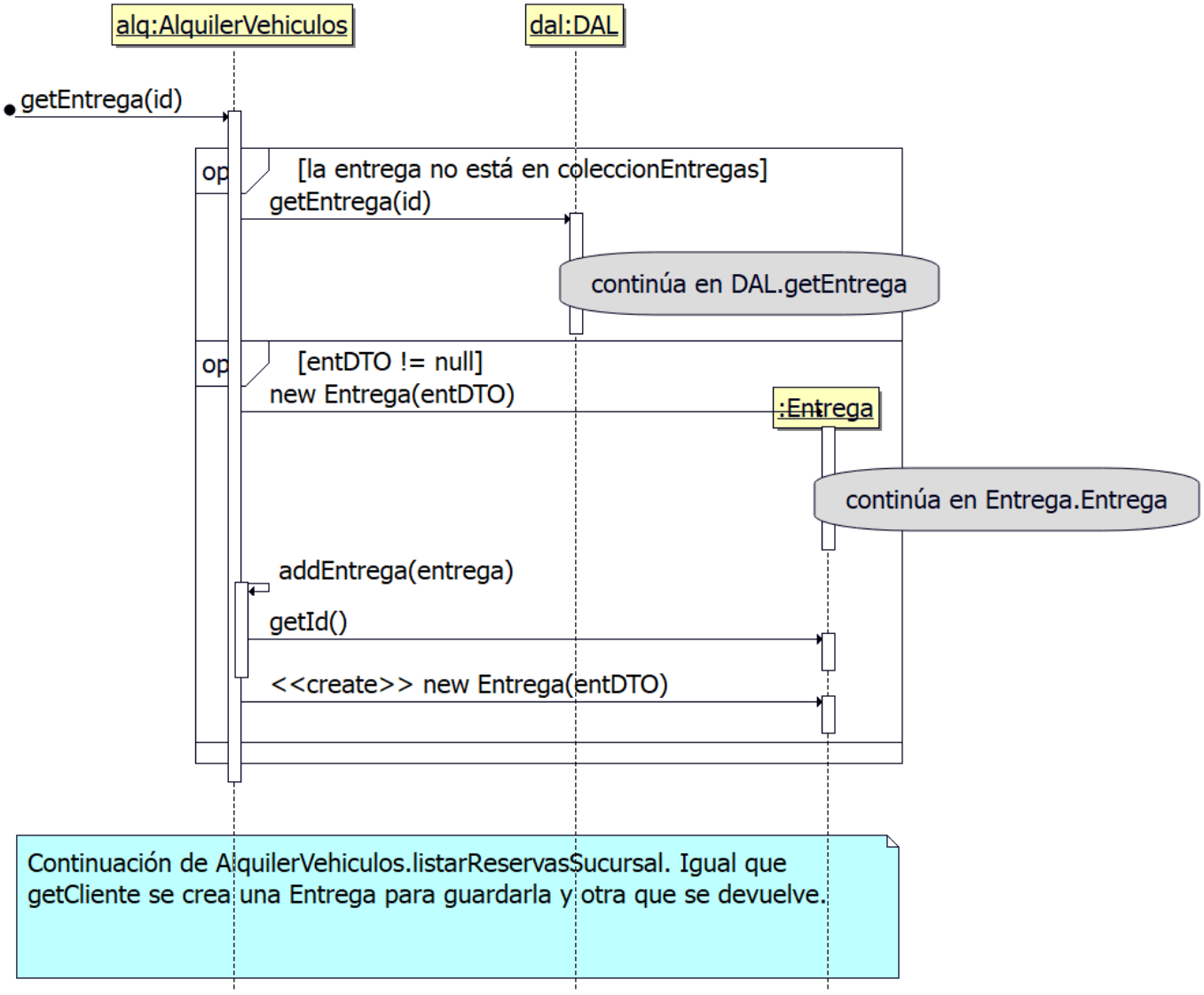


Figura 13. AlquilerVehiculos.getEntrega.

AlquilerVehiculos.listarVehiculosCategoriaSucursal

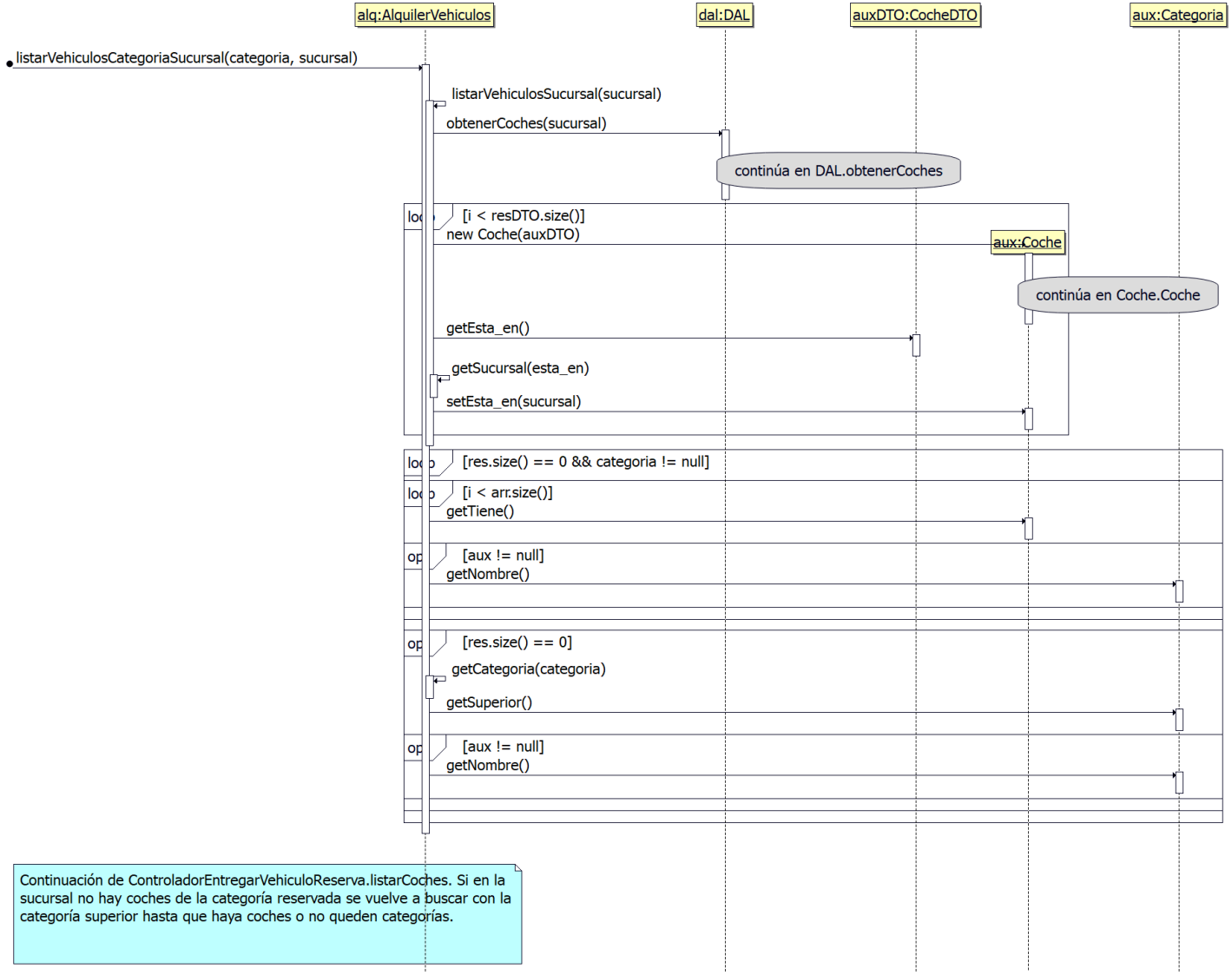


Figura 14. AlquilerVehiculos.listarVehiculosCategoriaSucursal.

AlquilerVehiculos.getEmpleado

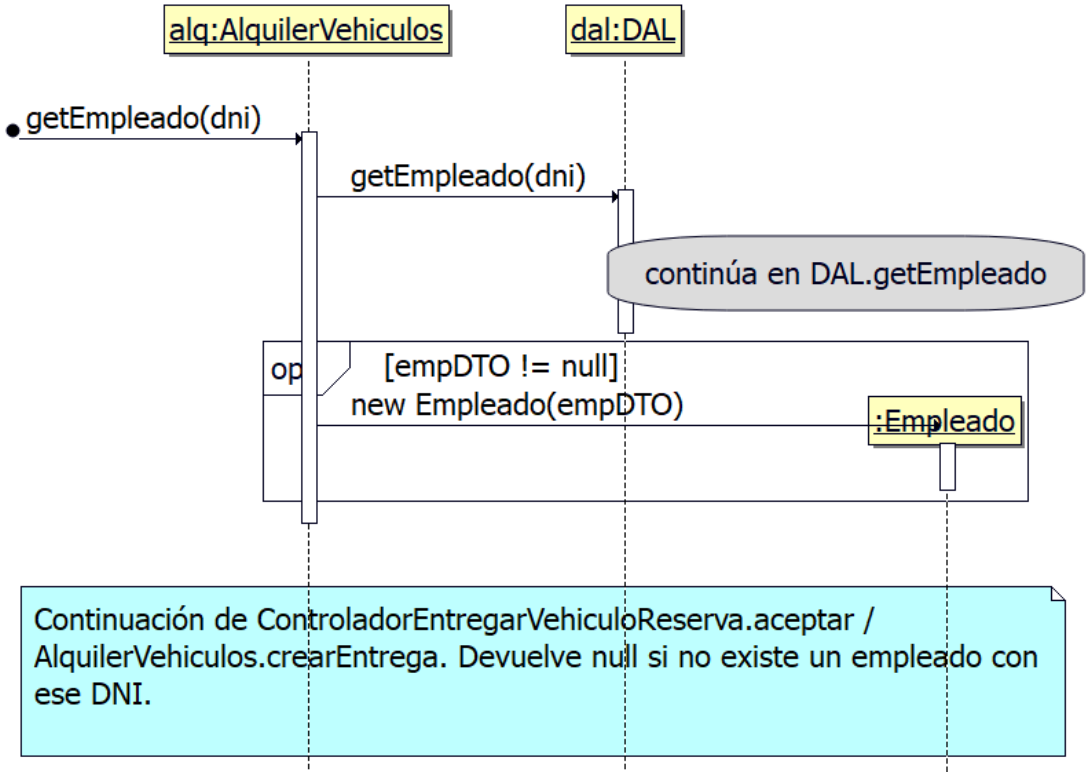


Figura 15. AlquilerVehiculos.getEmpleado.

AlquilerVehiculos.crearEntrega

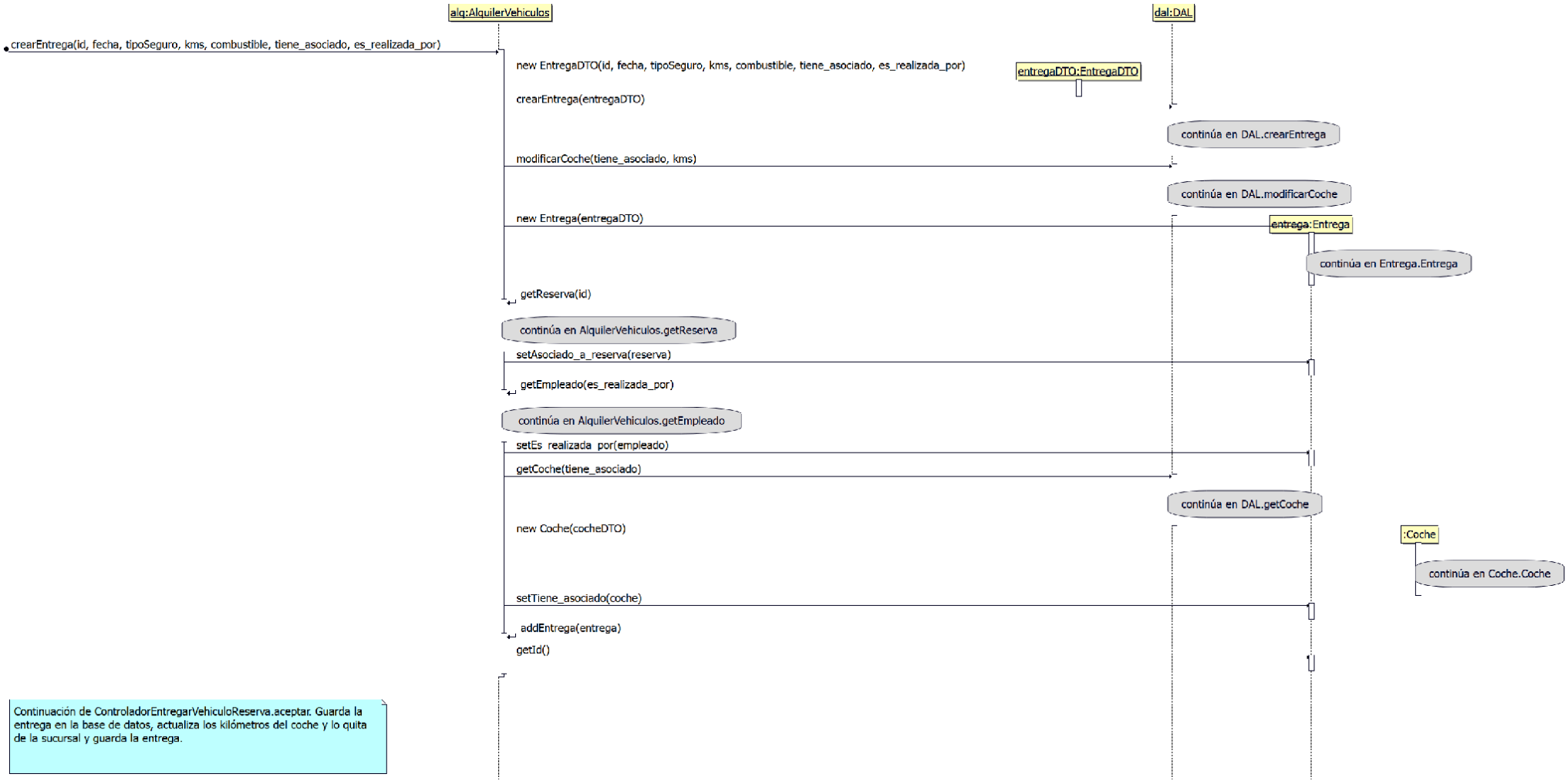


Figura 16. AlquilerVehiculos.crearEntrega.

AlquilerVehiculos.getReserva

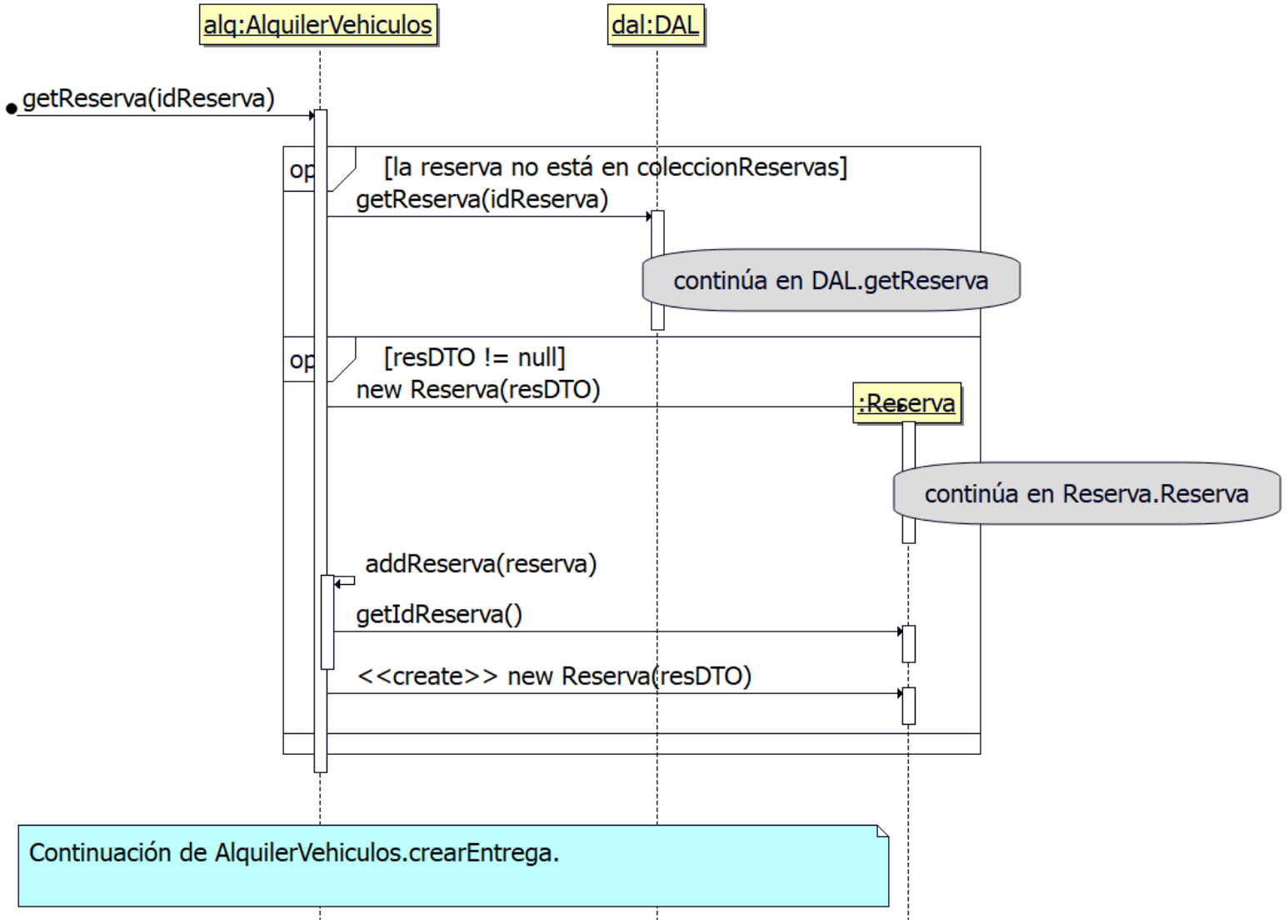


Figura 17. AlquilerVehiculos.getReserva.

Continuaciones de los constructores

Reserva.Reserva

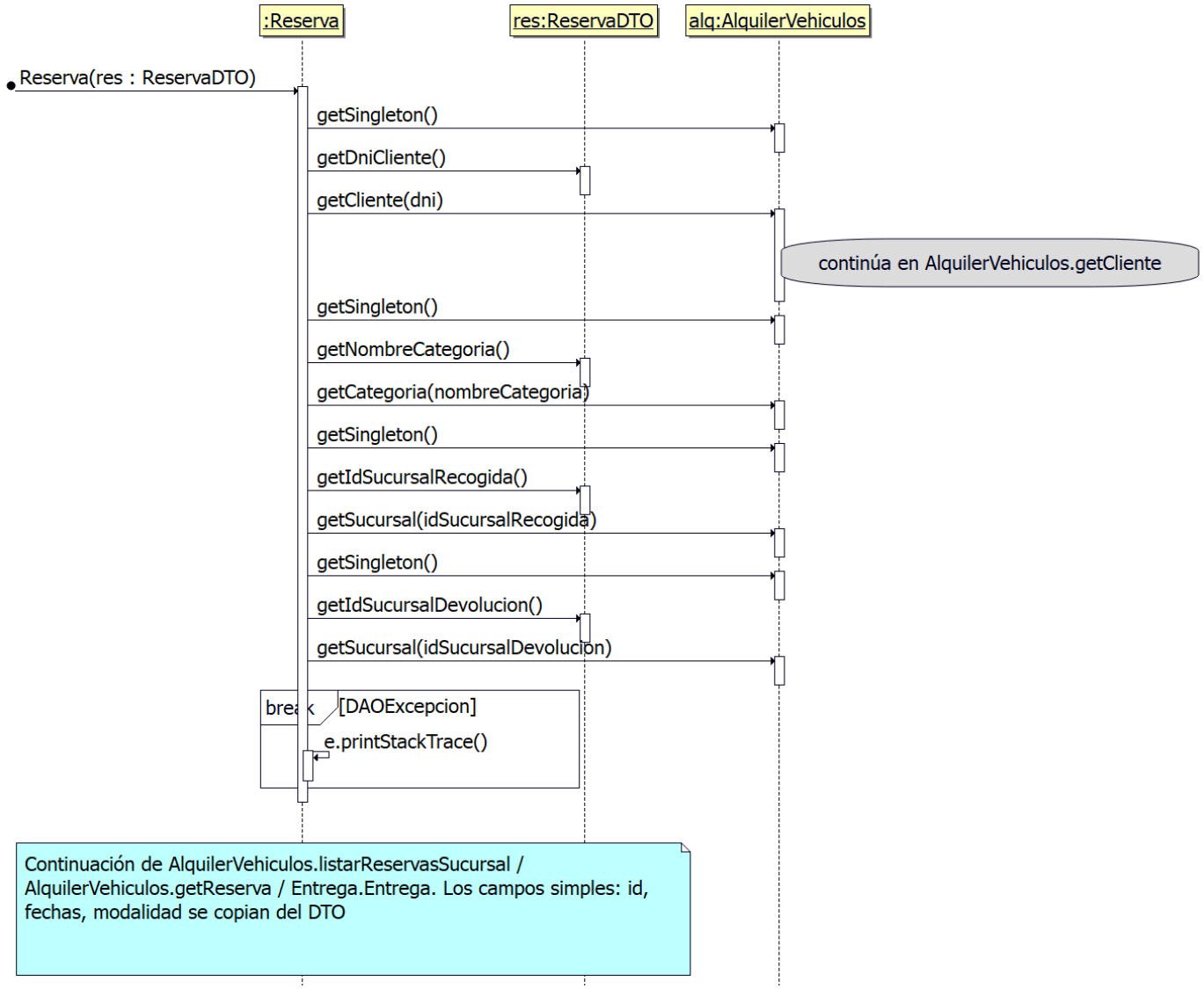


Figura 18. Reserva.Reserva.

Entrega.Entrega

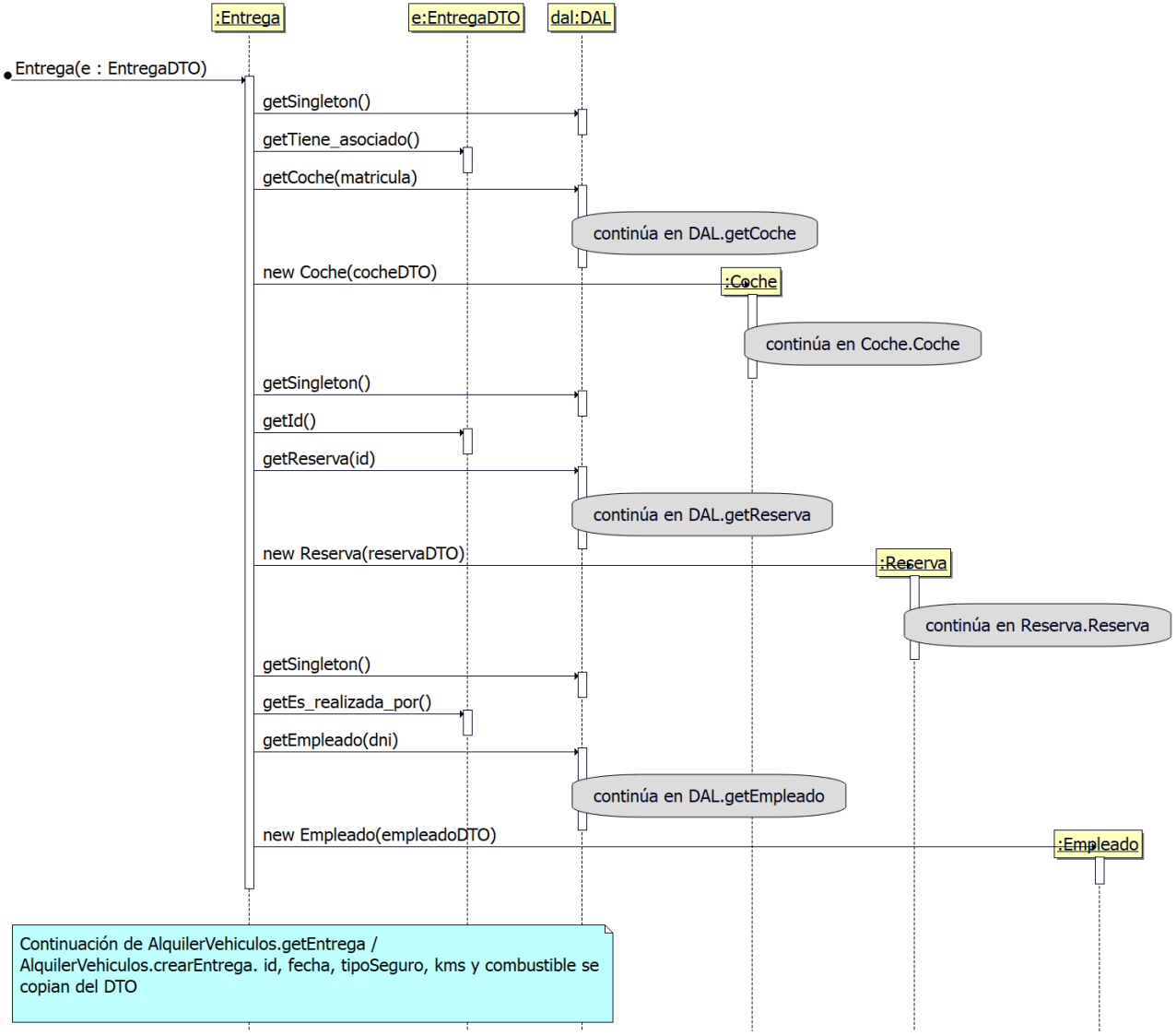


Figura 19. Entrega.Entrega.

Coche.Coche

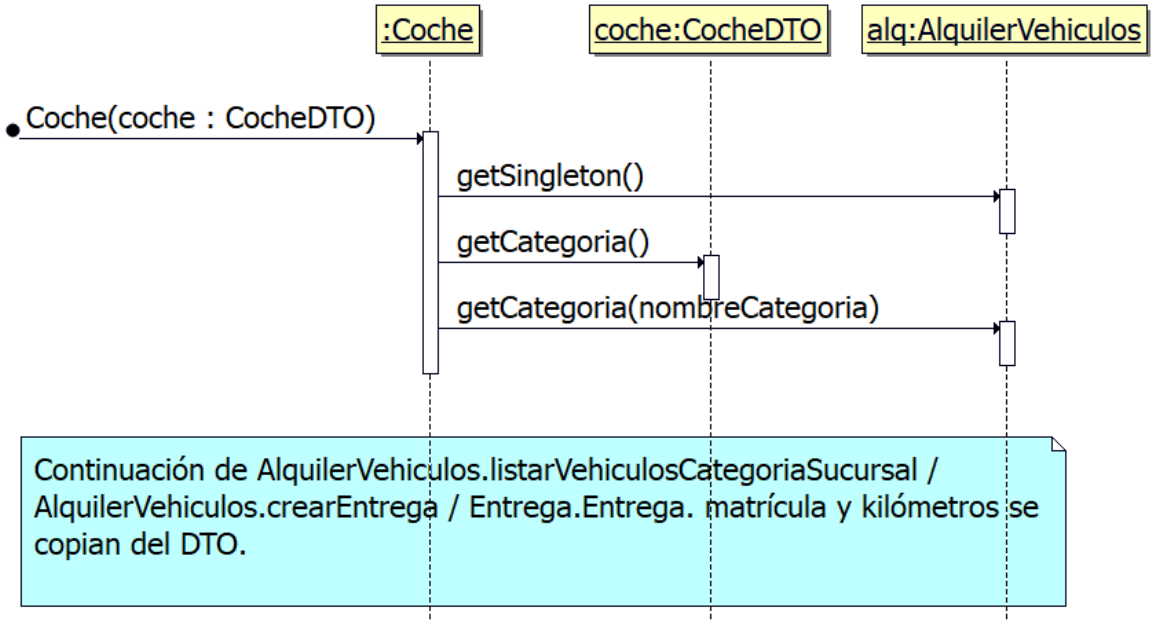


Figura 20. Coche.Coche.

Continuaciones de persistencia

DAL.getReservasSucursal

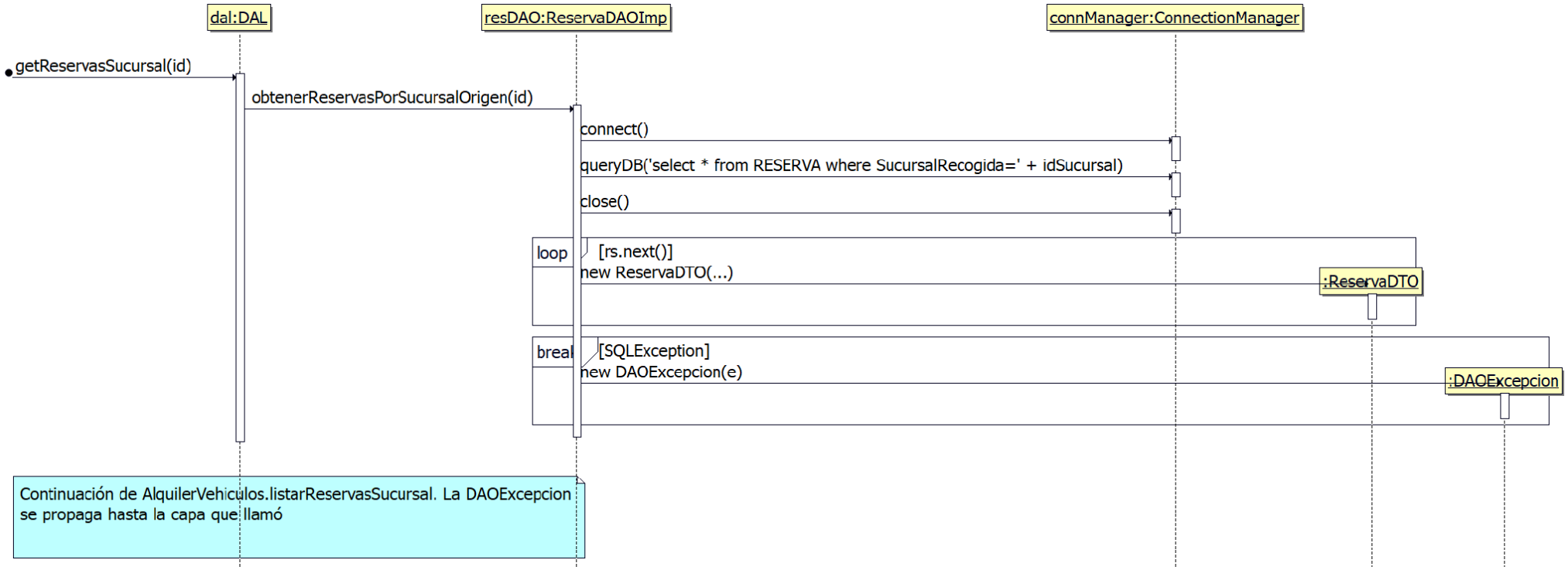


Figura 21. DAL.getReservasSucursal.

DAL.getCliente

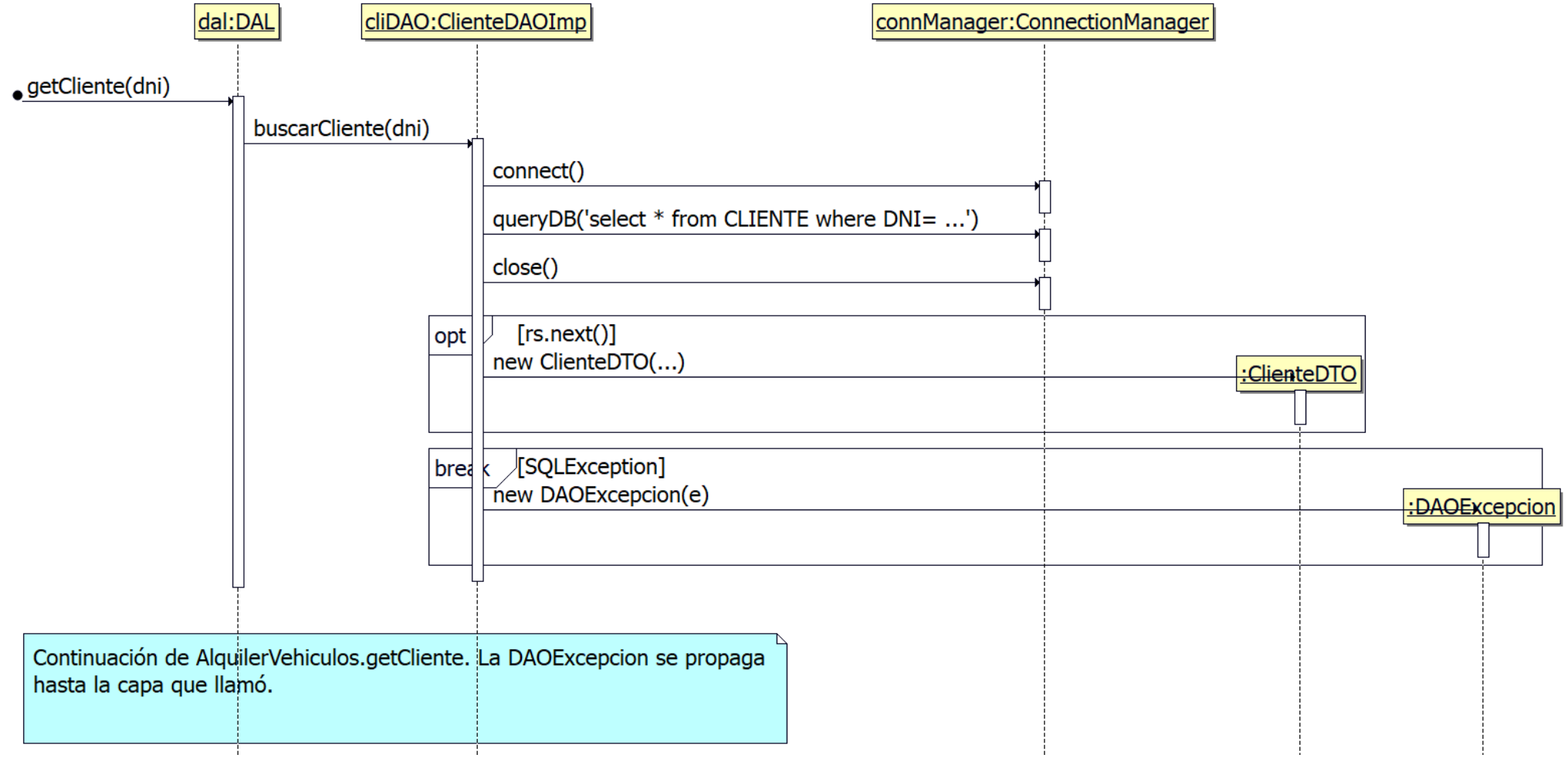


Figura 22. DAL.getCliente.

DAL.getEntrega

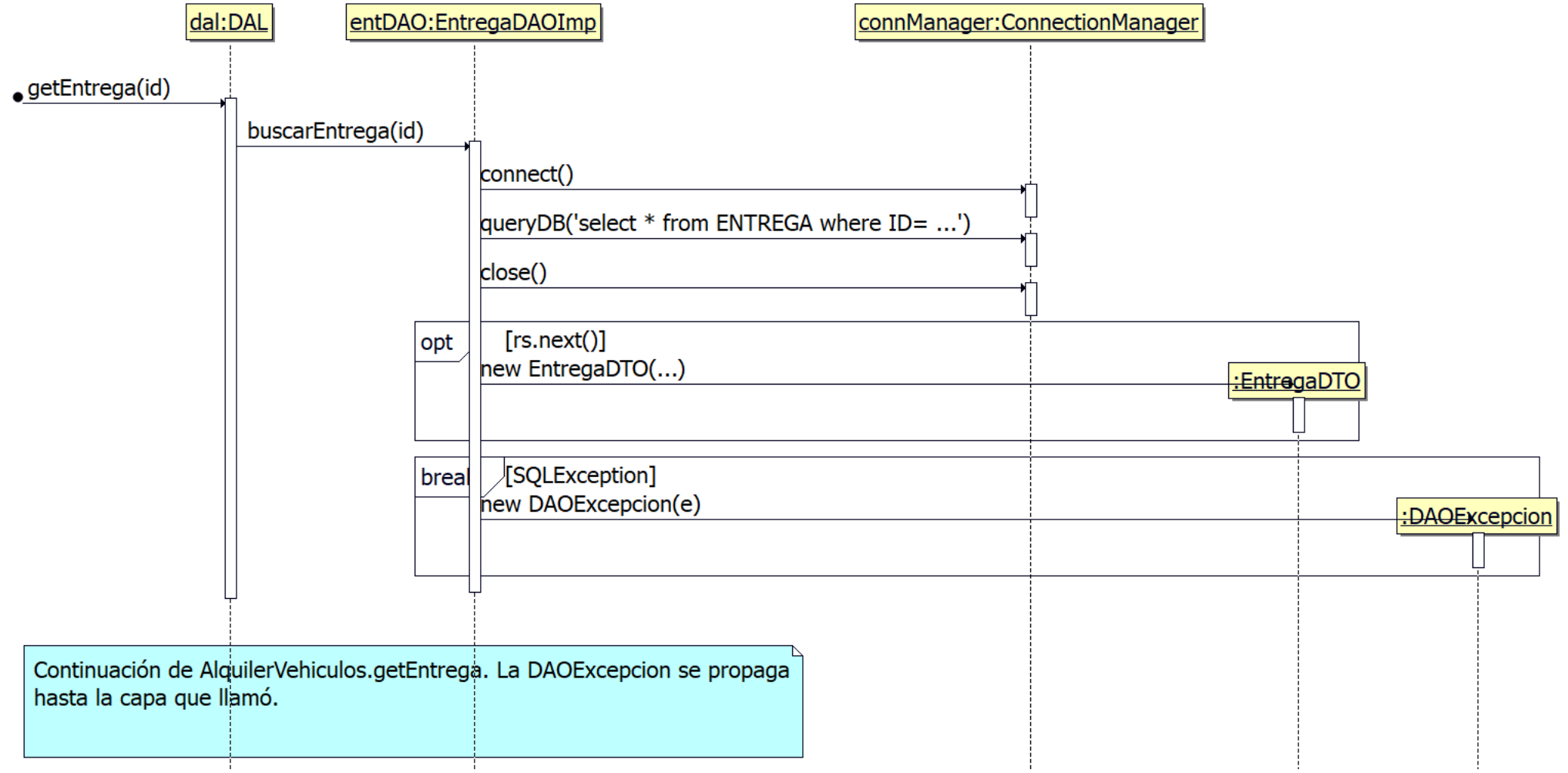


Figura 23. DAL.getEntrega.

DAL.obtenerCoches

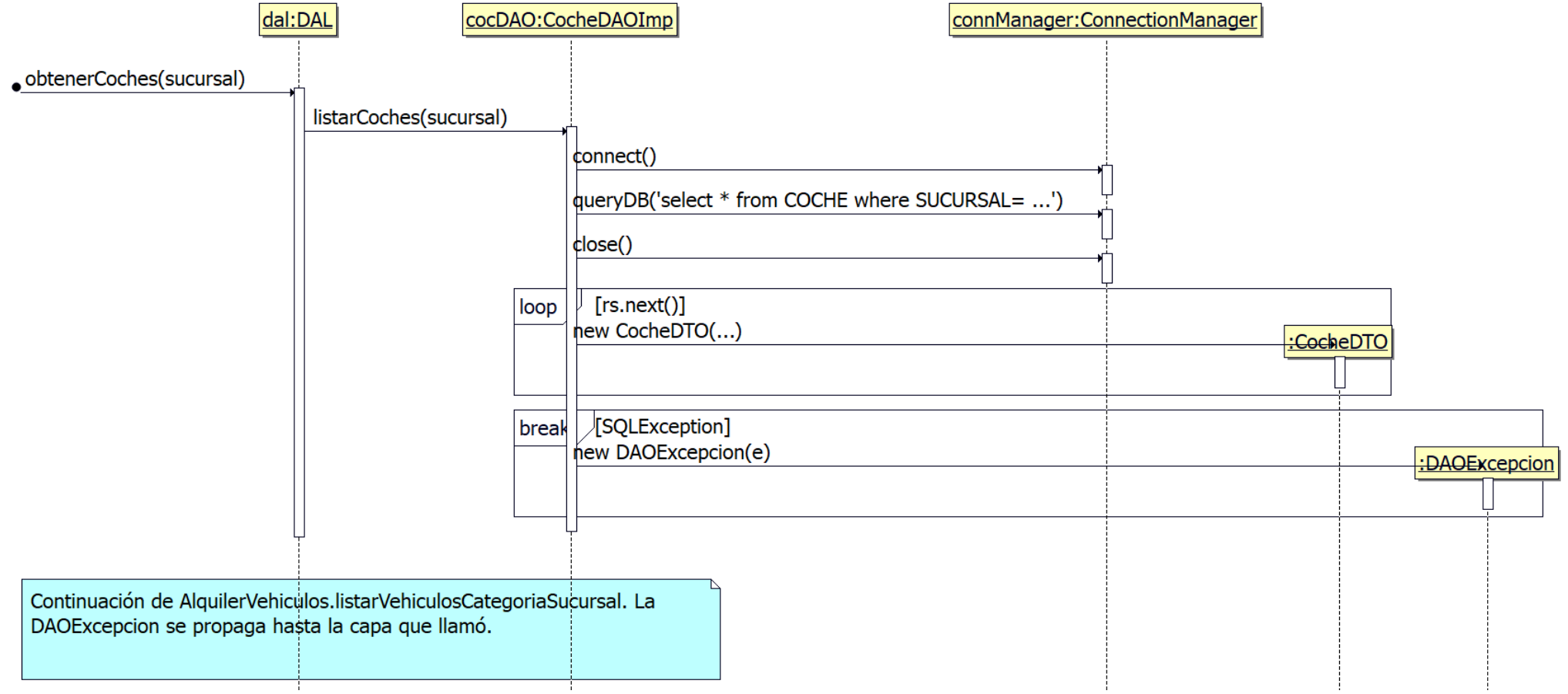


Figura 24. DAL.obtenerCoches.

DAL.getEmpleado

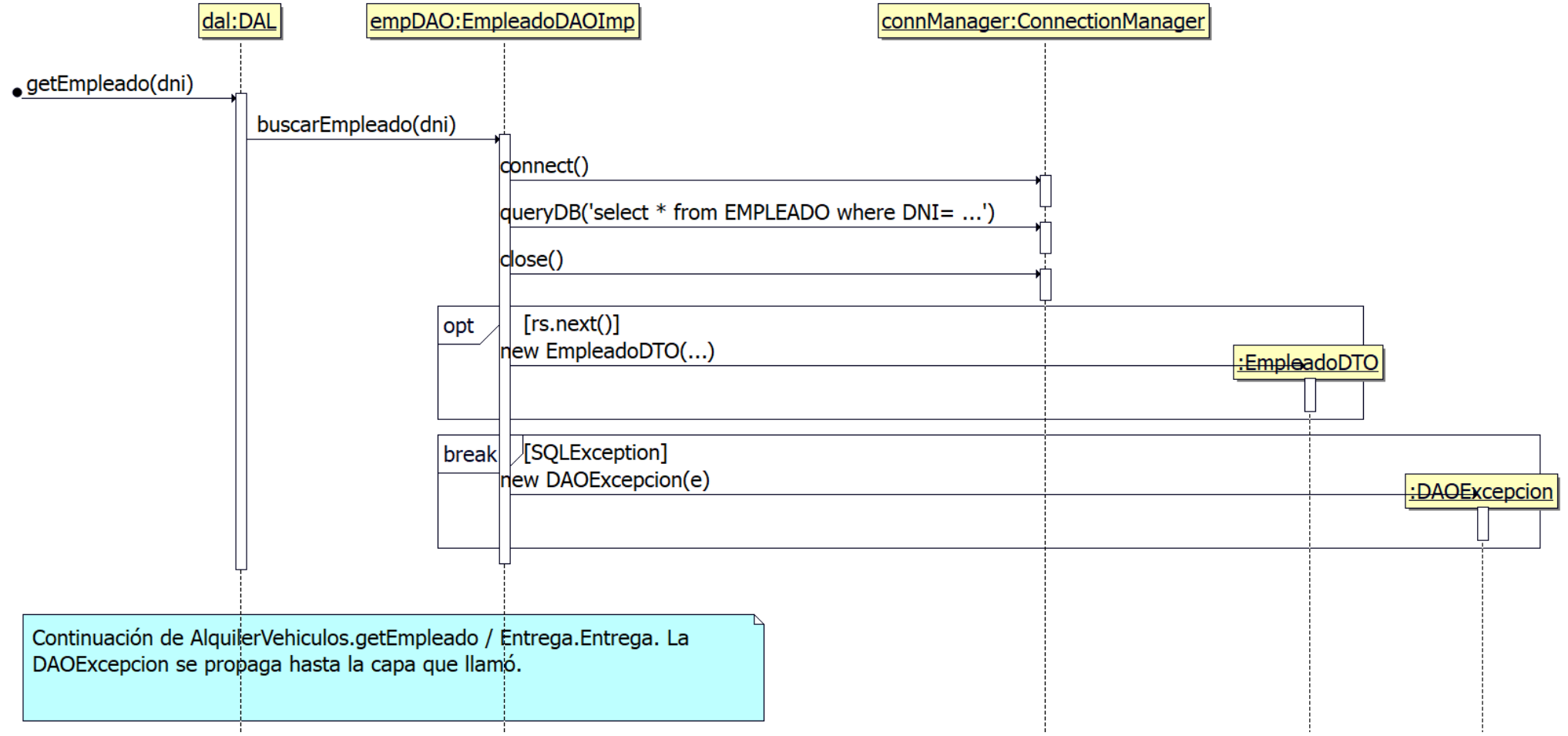


Figura 25. DAL.getEmpleado.

DAL.getCoche

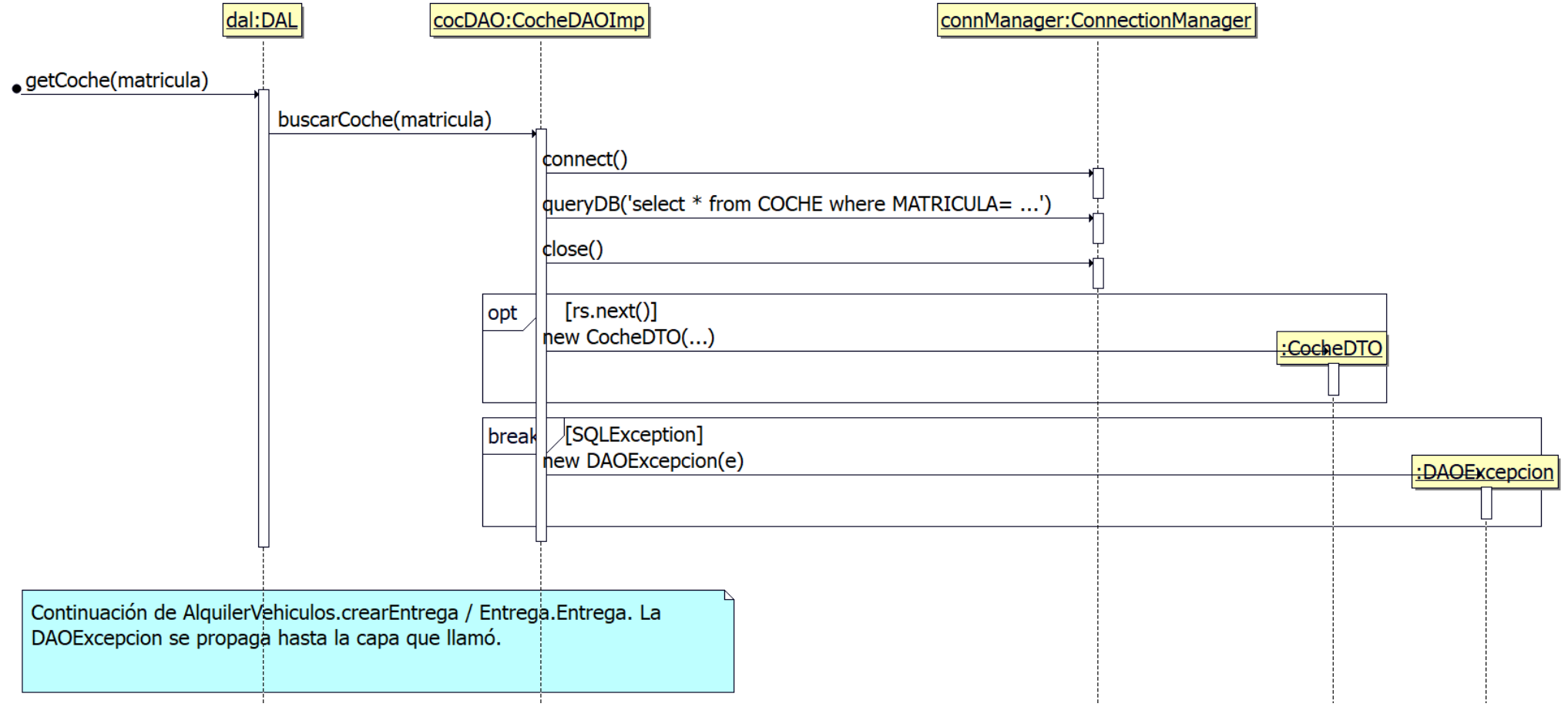


Figura 26. DAL.getCoche.

DAL.getReserva

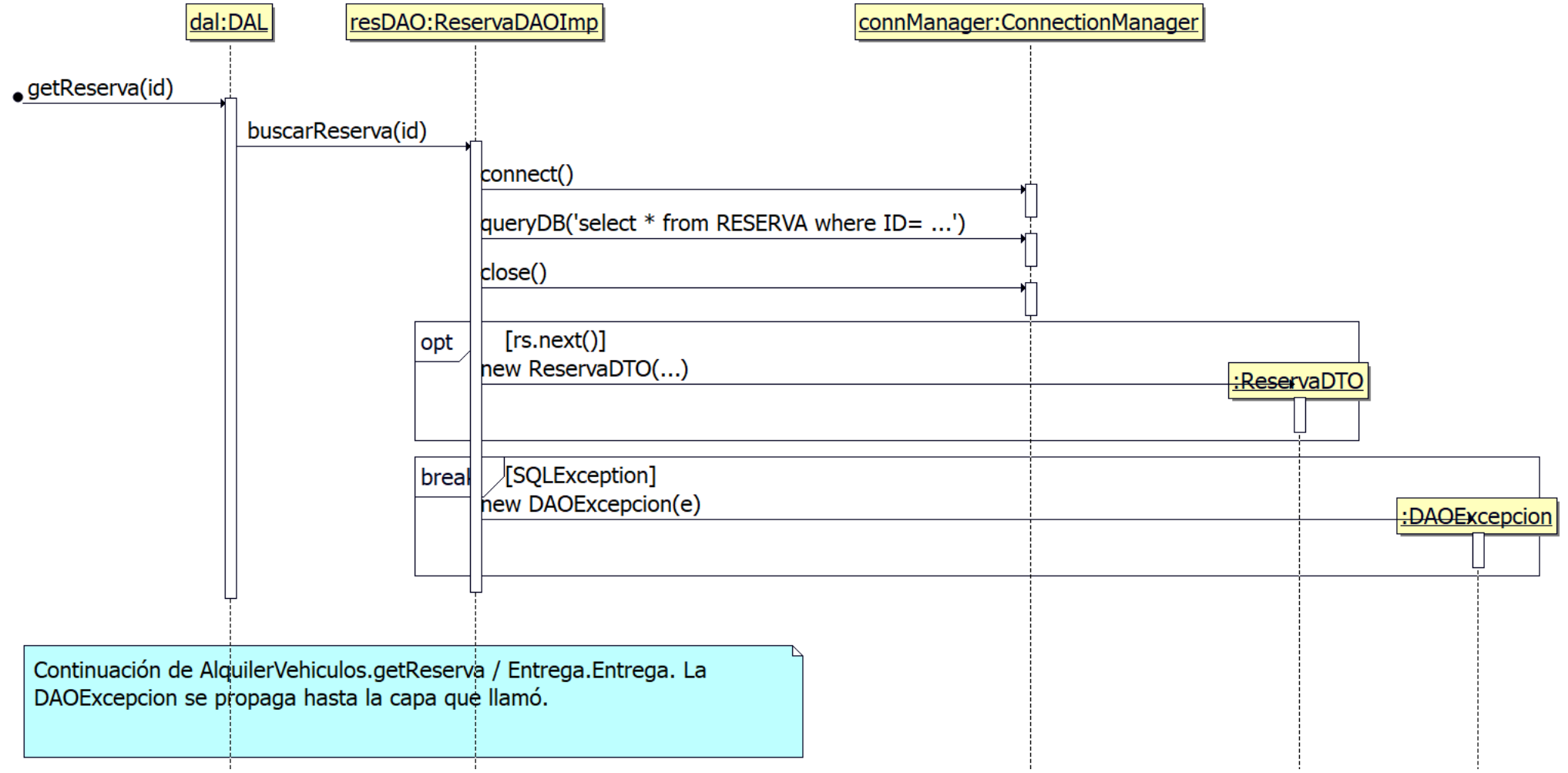


Figura 27. DAL.getReserva.

DAL.modificarCoche

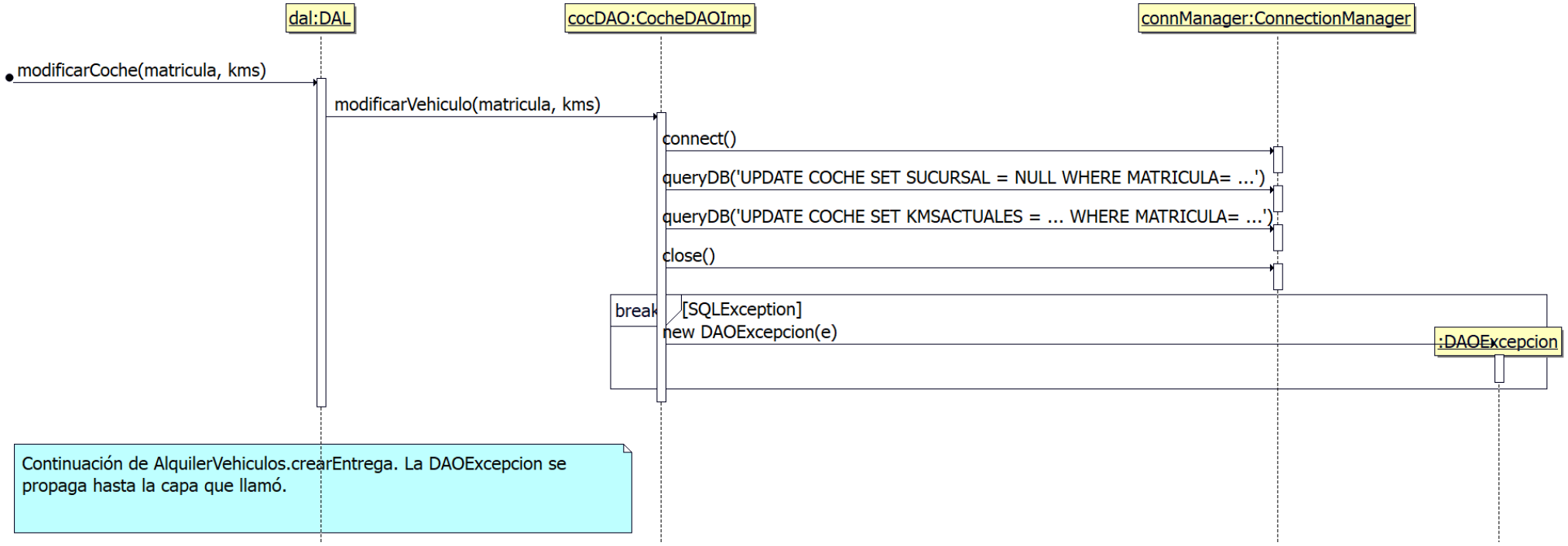


Figura 28. DAL.modificarCoche.

DAL.crearEntrega

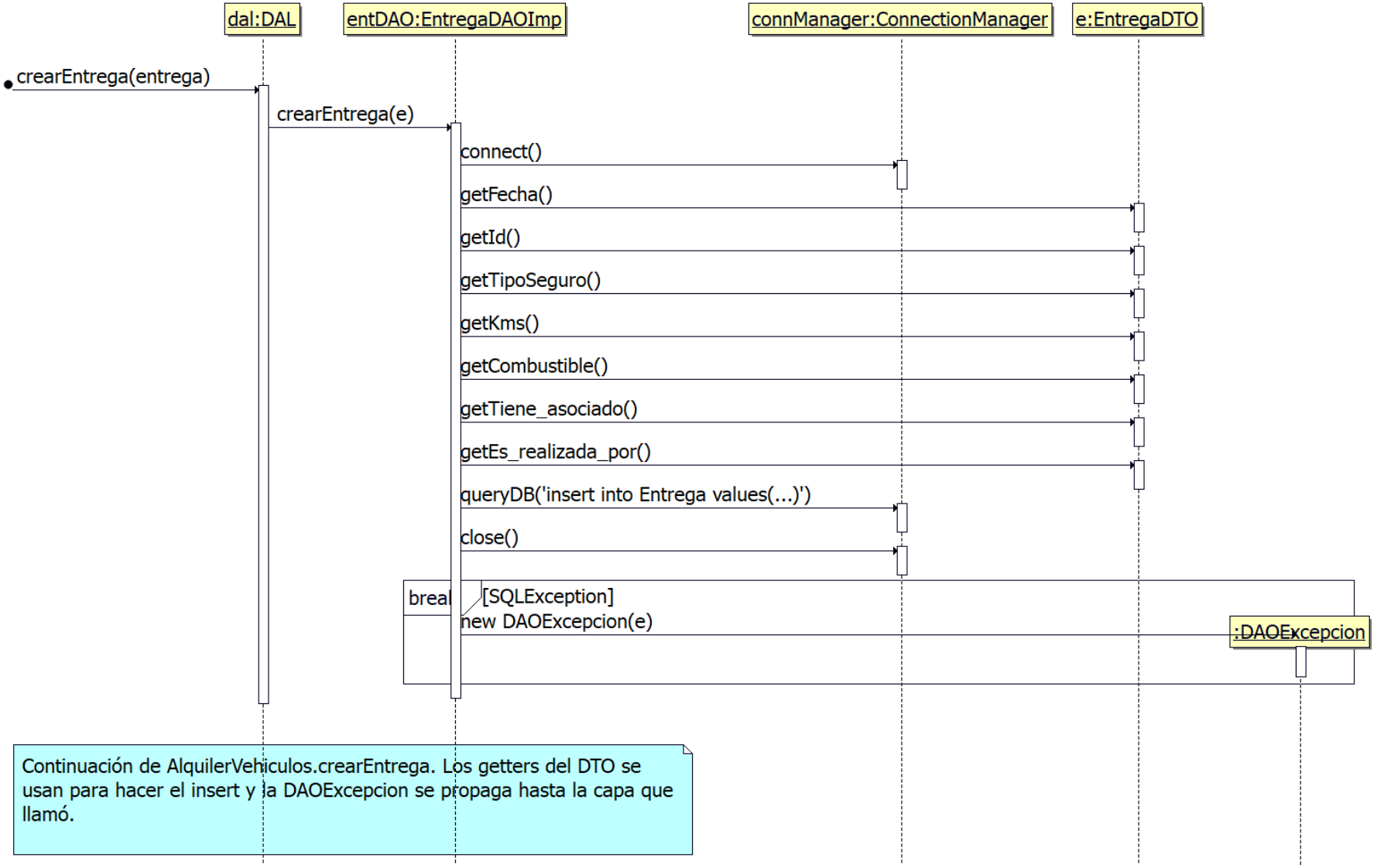
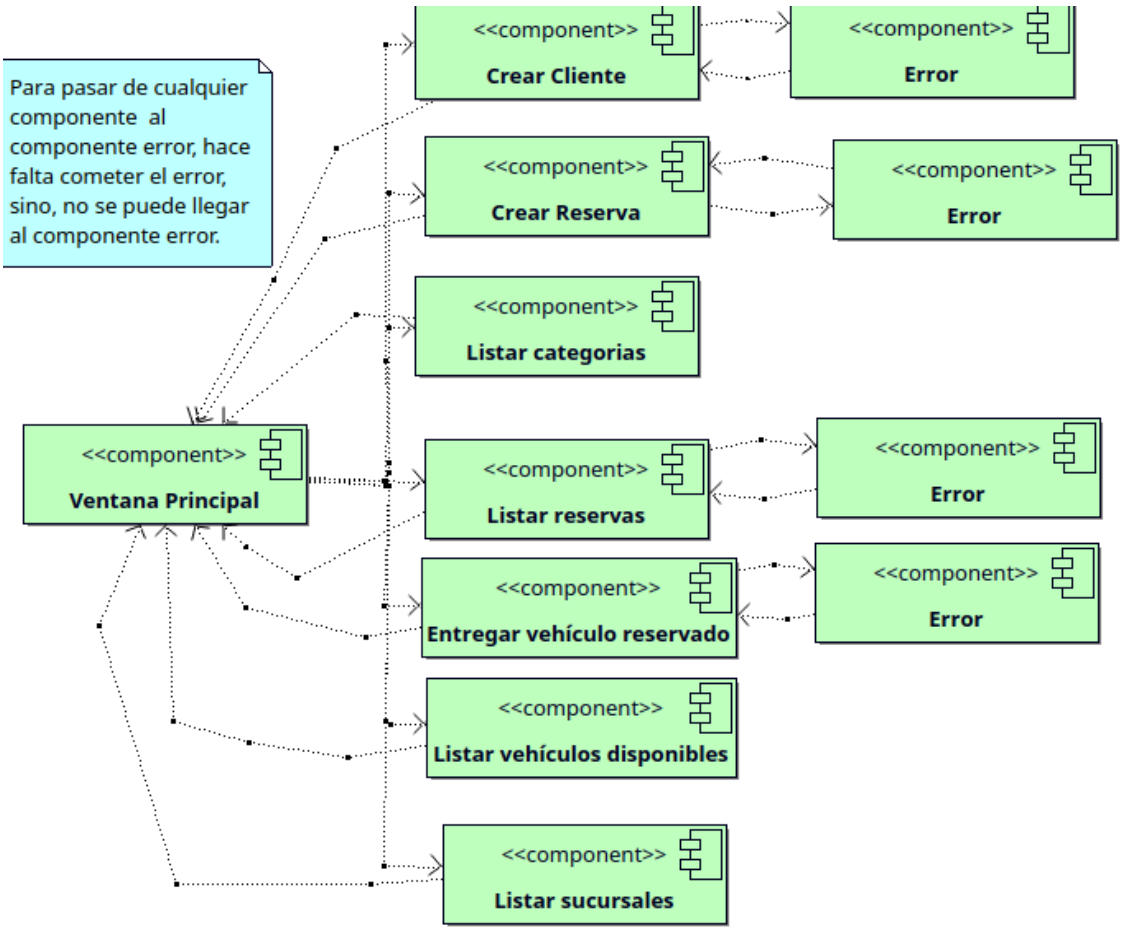


Figura 29. DAL.crearEntrega.

3 Mapa de la interfaz gráfica de usuario

El mapa de la interfaz gráfica de usuario presenta una estructura sencilla. La aplicación cuenta con una 'Ventana Principal', desde la cual es posible navegar hacia diversas páginas: 'Crear Cliente', 'Crear Reserva', 'Listar Categorías', 'Listar Reservas', 'Entregar Vehículo Reservado', 'Listar Vehículos Disponibles' y 'Listar Sucursales'. La navegación entre la ventana principal y estos componentes es bidireccional, permitiendo al usuario regresar al inicio en cualquier momento.

Adicionalmente, ciertos componentes ('Crear Cliente', 'Crear Reserva', 'Listar Reservas' y 'Entregar Vehículo Reservado') incluyen manejo de errores. Si se produce un fallo en alguna de estas páginas, el sistema redirige al usuario a una ventana de 'Error'. Al cerrar dicha ventana de error, la aplicación devuelve al usuario al componente anterior donde se originó el problema.



4 Declaración de diligencia en el uso de la IA

4.1 Ejercicios 2.2.1 y 2.2.2

Esta sección recoge el uso de IA en los ejercicios de clases y secuencia del caso de uso Entregar vehículo reservado.

Claude Opus 5.5

Según los borradores de trabajo, se utilizó para revisar las clases participantes y las notas del diagrama, así como para reorganizar su disposición y dividirlo en cuatro vistas. También se utilizó para la revisión de los diagramas de secuencia del segundo caso de uso.

ChatGPT con Codex y GPT-6

Se utilizó para la revisión de la memoria en Word, contrastar las explicaciones de los borradores con los diagramas y cuerpos Java guardados en A7.EJ2y3 y ayuda mediante una guía con pasos sobre cómo incorporar las exportaciones directas de BoUML. El resultado es esta memoria de los dos ejercicios.

Material consultado

Enunciado y presentación de la Práctica 3 de Ingeniería Inversa. Casos de uso del alquiler de vehículos. Proyecto A7.EJ2y3, vista EJ3. Borradores Explicacion_EJ2.2.1 y Explicacion_EJ2.2.2.

4.2 Ejercicio 3

Gemini 3.1 Pro

Se utilizó para la revisión y mejora del texto usado en dicho apartado.

Material consultado

Enunciado y presentación de la Práctica 3 de Ingeniería Inversa. Proyecto A7.EJ2y3, vista EJ3.

4.3 Aportaciones del resto del equipo

Falta completar. *Añadir, por ejercicio, la herramienta y versión utilizadas, para qué y por qué se usaron, los prompts relevantes y los resultados obtenidos.*